Generative Satellite Vision for Simulating Future Urban Morphology under Climate Pressure

SWE3004 - Dissertation-I

Submitted in partial fulfillment of the requirements for the degree of

MTech (Software Engineering)

in

Department of Software and Systems Engineering

by

GOUTHAM K

21MIS0386

Under the guidance of

Dr. JAGANNATHAN.J

School of Computer Science Engineering and Information Systems

VIT, Vellore

November 2025

DECLARATION

I hereby declare that the Dissertation-I entitled "Generative Satellite Vision for Simulating Future Urban Morphology Under Climate Pressure" submitted by me, for the award of the degree of *MTech (Software Engineering) in Department of Software and Systems Engineering, School of Computer Science Engineering and Information Systems* to VIT is a record of Bonafide work carried out by me under the supervision of Dr. JAGANNATHAN .T, Assistant Professor Sr.Grade 1, SCORE, VIT, Vellore.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date:

Signature of the Candidate

CERTIFICATE

This is to certify that the Dissertation-I entitled "Generative Satellite Vision for Simulating Future Urban Morphology Under Climate Pressure" submitted by **Goutham K 21MIS0386**, SCORE, VIT, for the award of the degree of *M.Tech (Software Engineering) in Department of Software and Systems Engineering*, is a record of Bonafide work carried out by him / her under my supervision during the period, 9.7.2025 to 14.11.2025, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or fully, for the award of any other degree or diploma in this institute orany other institute or university. The dissertation fulfills the requirements and regulations ofthe University and in my opinion meet the necessary standards for submission.

Place: Vellore

Date:

Signature of the VIT-SCORE - Guide

Internal Examiner **External Examiner**

Head of the Department

Department of Software and Systems Engineering

ACKNOWLEDGEMENT

It is my pleasure to express with a deep sense of gratitude to my **Dissertation- I** guide Dr. Jagannathan. J, Assistant Professor Sr.Grade 1, School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore for **his** constant guidance, continual encouragement, in my endeavor. My association with **him** is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and an expert in the field of AI & ML

"I would like to express my heartfelt gratitude to Honorable Chancellor **Dr. G Viswanathan**; respected Vice Presidents **Mr. Sankar Viswanathan**, **Dr. Sekar Viswanathan**, Vice Chancellor **Dr. V. S. Kanchana Bhaaskaran**; Pro-Vice Chancellor **Dr. Partha Sharathi Mallick**; and Registrar **Dr. Jayabarathi T**.

My whole-hearted thanks to Dean **Dr. Daphne Lopez**, School of Computer Science Engineering and Information Systems, Head, Department of Software and Systems Engineering **Dr. Thanapal P**, M.Tech (SE) Project Coordinators **Dr. Thilagavathi M** , **Dr. Brijendra Singh,** SCORE School Project Coordinator **Dr. Thandeeswaran R**, all faculty, staff and members working as limbs of our university for their continuous guidance throughout my course of study in unlimited ways.

It is indeed a pleasure to thank my parents and friends who persuaded and encouraged me to take up and complete my **dissertation** successfully. Last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly towards the successful completion of the **dissertation**.

Place: Vellore

Date: **Goutham K**

Executive Summary

This project, *Generative Satellite Vision for Simulating Future Urban Morphology Under Climate Pressure*, addresses the urgent need for climate-resilient urban planning in rapidly growing cities. With rising temperatures, flooding, heat-island intensification, and unplanned expansion becoming increasingly common, traditional static models fail to capture the complex interaction between climate change and urban development. The proposed solution integrates satellite vision, climate projections, and advanced generative AI to simulate future city morphologies under different environmental stress scenarios, enabling planners to explore sustainable pathways.

The research introduces a three-layer AI-driven framework combining high-resolution satellite imagery, GIS datasets, and climate indicators such as heat stress, rainfall anomalies, and sea-level rise. The system leverages generative models—GANs and diffusion models—to produce realistic future urban scenarios. These simulations are validated with historical data and presented through an interactive web-based dashboard that allows policymakers and researchers to compare "what-if" scenarios, evaluate vulnerabilities, and explore adaptation strategies such as green infrastructure and controlled urban densification.

The project contributes a scalable and innovative tool that bridges the gap between urban planning and climate forecasting. By fusing Earth Observation data with generative AI, it enables predictive, scenario-based modelling of future cities—offering actionable insights for sustainable development, disaster preparedness, and long-term climate resilience. This framework lays the foundation for next-generation smart city planning and demonstrates how AI can support evidence-based decision-making in an era of accelerating climate challenges

CONTENTS **Page No.**

	Acknowledgement	iv
	Executive Summary	v
	Table of Contents	vi
	List of Figures	viii
	Abbreviations	x
1	**INTRODUCTION**	1
	1.1 Objective	2
	1.2 Motivation	2
	1.3 Background	3
2	**DISSERTATION / INTERNSHIP DESCRIPTION AND GOALS**	**4**
	2.1 Literature Survey	4
	2.2 Research Gap	8
	2.3 Objectives	9
	2.4 Problem Statement	10
	2.5 Project Plan	11
3	**TECHNICAL SPECIFICATION**	**12**
	3.1 Requirements	12
	3.1.1 Functional Requirements	12
	3.1.2 Non-Functional	13

3.2 Feasibility Study 14

3.2.1 Technical Feasibility 14

3.2.2 Economic Feasibility 16

3.2.3 Social Feasibility 18

3.3 System Specifications 20

3.3.1 Hardware Specification 20

3.3.2 Software Specification 22

4 **DESIGN APPROACH AND DETAILS (as applicable) 24**

4.1 System Architecture 25

4.2 Design 27

4.2.1 Use Case Diagram 27

4.2.2 Class Diagram 29

5 **METHODOLOGY 33**

6 **PROJECT DEMONSTRATION 39**

6.1 Sample Codes 43

7 **RESULT & DISCUSSION 52**

8 **SUMMARY 59**

9 **REFERENCES 60**

List of Figures

Figure No.	Title	Page No
Fig 2.5.1	Representing project Plan	11
Fig 4.1.1	System Architecture	25
Fig 4. 2.1.1	Use Case Diagram	27
Fig 4.2.2.1	Class Diagram	29
Fig 6.1.1	Successful import of required Python libraries and initialization of the Google Earth Engine environment.	43
Fig 6.1.2	Processing workflow for generating NDVI layers and mapping vegetation health	44
Fig 6.1.3	Initialization of preprocessing modules including NumPy, Pandas, RasterIO, GeoPandas, OpenCV, and Scikit-Learn.	45
Fig 6.1.4	Data quality assessment workflow evaluating pixel validity, completeness, and descriptive statistics for multi-band satellite inputs.	46
Fig 6.1.5	Processing pipeline showing original RGB image, detected cloud mask, and final cloud-corrected RGB output after applying advanced masking and OpenCV inpainting.	47
Fig 6.1.6	Definition of the Ultra-Fast GAN architecture, including the Generator and Discriminator built using transposed and standard convolutional layers	48
Fig 6.1.7	Code for generating multiple future urban scenarios using the Ultra-Fast Generator and visualizing the output maps.	49
Fig 6.1.8	FastAPI backend setup integrating multiple route modules and configuring the GENURBAN API with CORS, endpoints, and server initialization.	50
Fig 6.1.9	Directory structure of the GENURBAN frontend built using React, Vite, and TailwindCSS.	51

Fig 7.1.1	Model-generated spatial simulations illustrating four alternative future urban morphology patterns.	57
Fig 7.1.2	Analysis view of Lucknow in GenUrban displaying density, vegetation, and built-up indicators with scenario controls.	58
Fig 7.1.3	AI-driven scenario generation results for Lucknow, showing diverse planning outcomes.	58

List of Abbreviations

AI	Artificial Intelligence
GAN	Generative Adversarial Network
GIS	Geographic Information System
RS	Remote Sensing
ML	Machine Learning
EO	Earth Observation
UHI	Urban Heat Island
LST	Land Surface Temperature
NDVI	Normalized Difference Vegetation Index
SLR	Sea-Level Rise

CHAPTER 1

INTRODUCTION

Urbanization has emerged as a defining global trend of the 21st century, with cities expanding rapidly to support population growth and economic development. However, this expansion has intensified environmental stress, leading to issues such as rising temperatures, worsening air quality, and increased vulnerability to natural disasters. Climate change further amplifies these risks, causing frequent extreme weather events and sea-level rises that threaten urban stability. Traditional urban planning models, which rely on static data, fail to capture these evolving complexities. As a result, there is a growing need for dynamic, data-driven approaches to understand how cities may respond to future climate pressures. This highlights the urgency for innovative tools capable of forecasting sustainable urban pathways.

Recent advancements in artificial intelligence, generative modelling, and satellite-based Earth observation have created new opportunities for predictive urban analysis. High-resolution satellite imagery, when combined with climate projections, offers a powerful means of understanding spatial and environmental changes over time. Generative AI techniques such as GANs and diffusion models can simulate realistic future urban landscapes under varying climate scenarios. These models go beyond prediction by generating alternative possibilities, enabling planners to explore resilient design strategies. Such capabilities support data-backed decision-making in areas like infrastructure development, risk mitigation, and sustainable resource allocation. Through scenario-based simulations, cities can better prepare for climate uncertainties.

This project introduces a Generative Satellite Vision framework that uses satellite imagery, climate projections, and generative AI to simulate future urban morphology. The system generates realistic scenario-based visualizations to assess climate risks such as heat islands, flooding zones, and urban sprawl. By integrating multi-source geospatial data with GANs and diffusion models, it provides adaptive and data-driven predictions for planners. Ultimately, the project delivers a decision-support tool for designing sustainable, climate-resilient cities.

1.1Objective

The primary objective of this project is to develop a Generative AI-driven framework that simulates future urban morphology under climate pressure using satellite imagery and climate projections. The system aims to integrate multi-source geospatial datasets into a unified pipeline that supports accurate and scalable urban simulations. By applying generative models such as GANs and diffusion models, the project seeks to produce realistic scenario-based urban growth patterns. Another key objective is to enable "what-if" climate explorations, allowing planners to evaluate the impact of heat stress, flooding, and sea-level rise. The framework also focuses on building an interactive visualization dashboard for intuitive scenario analysis. Additionally, the project aims to validate model outputs against historical data to ensure reliability. Ultimately, the objective is to support climate-resilient decision-making for sustainable and adaptive urban planning.

1.2 Motivation

Rapid urbanization and climate change are reshaping cities at an unprecedented pace, creating urgent challenges for planners and policymakers. Rising temperatures, increased flooding, and unplanned expansion are placing immense stress on existing infrastructures. Traditional models, which rely on static and isolated datasets, are no longer sufficient to predict complex urban–climate interactions. There is a growing need for dynamic tools that can forecast long-term urban transformation. This motivates the development of smarter, AI-driven approaches to understand future city scenarios.

Advancements in satellite vision and generative AI offer powerful opportunities to address these emerging challenges. High-resolution satellite imagery provides detailed insights into land use, vegetation loss, and population-driven expansion. When combined with climate projections, these datasets reveal how environmental pressures influence urban form. Generative models such as GANs and diffusion networks enable simulation of multiple possible futures instead of a single prediction. This creates a foundation for exploring sustainable planning strategies through scenario-based analysis.

This project is motivated by the need to equip cities with a predictive, adaptive, and science-backed decision-support system. By integrating Earth Observation data with generative AI, the framework empowers planners to visualize heat islands, flood-prone zones, and alternative growth pathways. The ability to test "what-if" climate scenarios help policymakers evaluate resilient infrastructure options. The project ultimately aims to guide cities toward climate-ready urban development. Its motivation lies in building safer, smarter, and more sustainable urban futures.

1.3 Background

Urbanization has accelerated across the world, leading to denser cities, rapid infrastructure expansion, and significant land-use changes. As cities continue to grow, they increasingly interact with climate systems, resulting in challenges such as rising surface temperatures, intensified urban heat islands, reduced green cover, and increased flooding risks. Traditional urban planning approaches rely on static datasets and linear models, which fail to capture the complex and nonlinear relationship between urban morphology and climate change. This gap has created a global need for advanced tools that can analyze, predict, and simulate how cities may evolve under varying environmental pressures.

Recent advancements in Earth Observation (EO) technologies, especially satellite-based remote sensing, have transformed how researchers study urban environments. Satellite datasets like Sentinel-2, Landsat, and climate-based indices such as NDVI and LST offer continuous, high-resolution information about land dynamics. Parallelly, the rise of artificial intelligence—particularly generative models such as GANs and diffusion networks—has unlocked the ability to create realistic future scenarios instead of relying only on historical data. Combined, these technologies enable simulation-based forecasting of urban growth patterns under projected climate conditions. However, existing research mostly analyzes past trends rather than generating possible future morphologies.

Existing research lacks generative AI frameworks that simulate future urban morphology under climate pressure. This project addresses that gap by integrating satellite data and climate projections to generate realistic scenario-based urban futures. Its goal is to provide a scalable tool that supports climate-resilient planning and decision-making

2.DISSERTATION DESCRIPTION AND GOALS

2.1 Literature Survey

A study on the influence of 2D/3D urban morphology on diurnal land surface temperature highlights how building height, density, and spatial patterns impact thermal variations. Using remote sensing datasets, the work demonstrates strong climate–morphology relationships. The findings help quantify heat island intensity based on structural form. However, generalization remains limited due to region-specific datasets.

Research using XG-Boost to analyze relationships between 2D/3D morphology and seasonal LST gradients demonstrates the power of machine learning for climate–urban studies. The model captures nonlinear dependencies between morphology and temperature changes across seasons. It enhances prediction accuracy compared to traditional statistical methods. Limitations arise from regional biases and feature dependencies.

A study exploring the impact of 2D/3D building morphology on LST across three megacities shows that built form significantly influences thermal behavior. Using satellite imagery, the authors highlight how different building structures modify land surface temperature. The findings emphasize the role of urban design in heat management. However, results remain case-specific and difficult to generalize globally.

Large-area characterization of urban morphology using TanDEM-X data provides accurate mapping of building height and density. This enables improved understanding of urban structure and climate impacts on the city scale. This method enhances morphological modelling through detailed elevation datasets. Yet, high data requirements and processing complexity limit widespread applicability.

Large-scale classification of urban structural units using remote sensing imagery demonstrates effective segmentation of diverse urban forms. Machine learning improves differentiation between built-up patterns and land uses. The system offers value for morphological mapping

and planning studies. Nonetheless, accuracy is strongly dependent on dataset quality and sensor characteristics.

Remote sensing-based attributes of urban heat islands identify key morphological contributors to UHI intensity. The study maps thermal anomalies and links them to urban design factors. It provides strong evidence for structural drivers of temperature variation. However, socioeconomic influences are not fully incorporated into the analysis.

Research correlating urban morphology with socioeconomic data shows how satellite-derived structural features reflect population and household characteristics. The findings highlight spatial inequality patterns linked to morphology. This approach expands the role of EO data in socioeconomic analysis. Yet, correlations do not confirm causation and may be biased by data limitations.

A study on the drivers of land surface temperature across urban functional zones reveals how zoning patterns influence heat variation. Using LCZ frameworks, the authors analyze temperature differences linked to urban function. This method enhances interpretation of thermal behavior within diverse urban sectors. Still, it requires heavy preprocessing and remains limited to specific regions.

A multi-source building-height estimation method provides a reliable way to derive structural elevation data without LiDAR. The approach combines several remote sensing inputs to improve height accuracy. It supports detailed morphological and climate studies in data-limited areas. However, accuracy heavily depends on auxiliary dataset availability.

Research on roles of 2D and 3D landscape metrics in the surface thermal environment identifies which morphological indicators most strongly affect LST. The study provides detailed analysis of height, density, and spatial variation metrics. It improves understanding of structural contribution to heat patterns. But validation is limited to selected urban regions.

A comparative study on UHI patterns across different urban morphologies examines consistency in heat island behavior. The authors show that morphology–UHI relationships vary by scale and context. This helps refine climate-sensitive design strategies. However, applicability to smaller or developing cities remains uncertain.

Work on built environment impacts on UHI in cold-climate cities highlights how climatic context changes morphological influence on temperature. The study identifies structural drivers specific to cold regions. It contrasts behavior with tropical and temperate cities. But findings remain restricted to cold-climate settings.

A convolutional autoencoder-based study analyzes morphological similarity across urban fabrics using high-dimensional remote sensing data. The model extracts latent features to cluster urban patterns effectively. This helps understand structural similarity across regions. Yet, interpretability of deep learning clusters remains a major limitation.

A study exploring influencing factors of LST integrates LCZ classification with climate and land-use data to analyze multi-variable temperature drivers. It highlights combined effects of morphology, vegetation, and climate indicators. This provides holistic understanding of urban heat formation. However, preprocessing complexity increases implementation difficulty.

An analysis of urban polycentric development using DMSP-OLS nighttime light data quantifies spatial growth centers across Chinese cities. The study identifies emerging sub-centers and development clusters. It offers insights into urban expansion and economic activity. But night-light data limitations cause low spatial resolution and potential biases.

Table 2.1 Merits and Demerits of Literature Survey

S. NO	TITLE	MERITS	DEMERITS
1	Gesture-Controlled Virtual Mouse And Finger Air Writing (2024 IEEE)	Supports both mouse control and air-writing; marker-free; improves accessibility.	Requires high processing power; accuracy decreases in poor lighting.
2	Air Mouse Using Bluetooth Technology (2024 IEEE)	Wearable and wireless; reduces physical strain; efficient for presentations.	Requires external hardware (wearable device); limited to Bluetooth range.
3	Virtual Mouse Control Using Colored Fingertips And Hand Gesture Recognition (2020 IEEE)	Simple to implement; enhances real-time cursor control.	Depends on colored markers; not robust under lighting changes.
4	Hand Gestures For Mouse Movements With Hand And Computer Interaction Model (2023 IEEE)	Provides natural hand–computer interaction; supports multiple gestures.	High dependency on consistent background; may lag in dynamic conditions.
5	Virtual Mouse Input Control Using Hand Gestures (2023 ICOEI)	Contactless operation; reduces dependency on physical devices.	Limited gesture set; may face calibration issues.
6	Hand Gesture Identification And Voice Command Based Hardware Reduction (2022 IEEE)	Combines gesture recognition with voice commands; reduces hardware needs.	Voice commands may fail in noisy environments; requires additional processing.
7	Gesture Recognition Based Virtual Mouse And Keyboard (IEEE Xplore)	Enables both mouse and keyboard control using gestures; versatile interaction.	Gesture complexity may confuse recognition; requires stable environment.
8	Gesture Recognition Based Virtual Mouse And	Robust HCI model; integrates keyboard features along with mouse.	Accuracy reduces with complex gestures; high computational demand.

	Keyboard (2020 ICOEI)		
9	Vision-Based Computer Mouse Control Using Hand Gestures (2015 IEEE	Simple vision-based model; reduces hardware dependency.	Sensitive to lighting variations; limited real-world adaptability.
10	Vision-Based Computer Mouse Control Using Hand Gestures (2015 IEEE)	Low-cost solution; accessible for general users.	Poor robustness; struggles with complex gestures and cluttered backgrounds.
11	Embedded Virtual Mouse System By Using Hand Gesture Recognition (2015 IEEE)	Achieves fingertip tracking with image processing; functional prototype.	Relies on colored markers; lacks flexibility in real-world use.
12	Virtual Mouse Using Hand Gesture (2023 IEEE)	Uses **MediaPipe** for marker-free recognition; high accuracy and low latency.	Computationally heavy; may struggle with fast gestures.
13	Mouse Cursor Control System Based on Hand Gesture (2015 Elsevier)	Uses contour and blob analysis; works with simple setups.	Sensitive to background noise and lighting; requires controlled environment.
14	A Survey on Hand Gesture Recognition for Simple Mouse Control (2015 IEEE)	Provides a comparative review of different techniques.	Survey only; does not implement or test a practical system.
15	A Survey of Fingertip Character Identification in Open-Air Using Image Processing and HCI (2018 IEEE)	Intuitive fingertip-based interaction; analyzes multiple systems.	Intuitive fingertip-based interaction; analyzes multiple systems.

2.2 Research Gap

Research in urban morphology and climate interaction has grown significantly, especially with the rise of satellite-based remote sensing and machine learning models. Most studies examine correlations between morphology, land surface temperature, and heat islands using 2D/3D metrics. While these works provide valuable environmental insights, they remain descriptive or predictive in nature. No existing research offers generative, scenario-based simulations for future urban evolution under climate pressure.

Existing studies still lack a unified framework that integrates multi-source satellite imagery, GIS layers, and climate projections for future urban simulation. Most papers rely on single-variable analyses, focusing only on LST, rainfall, or morphology without combining multiple climate stressors. Generative AI models such as GANs and diffusion models are completely absent in current urban morphology research. Existing approaches remain region-specific, limiting their ability to generalize across diverse cities and climates. Another major gap is the absence of scenario-based simulations that test "what-if" climate or policy interventions. Visualization dashboards for planners are rarely provided, leaving research outputs static and non-interactive. These gaps highlight the need for an adaptive, scalable, and generative framework for climate-resilient urban planning.

Overall, the literature shows strong analytical capability but lacks generative, adaptive tools. There is a need for a unified framework that merges satellite data, climate projections, and AI-driven simulations. This project aims to fill these gaps with a scalable, interactive, and climate-resilient urban simulation system.

2.3 Objectives

The primary objective of this project is to develop a Generative AI-driven framework that integrates satellite imagery, climate projections, and urban planning datasets to simulate future urban morphology under climate pressure. Unlike conventional static growth models, this

framework is designed to dynamically respond to multiple climate stressors. It aims to generate realistic, adaptive, and diverse future-city scenarios.

Key objectives of the project are:

Data Integration Pipeline: Develop a robust pipeline that fuses satellite imagery, GIS layers, and climate-change indicators (e.g., temperature rise, flood risk, air quality) into a unified, model-ready dataset for urban simulation.

Generative AI Modelling: Apply generative techniques (GANs and diffusion models) to simulate multiple urban growth trajectories that reflect both infrastructural expansion and environmental resilience.

Interactive Dashboard: Design an intuitive visualization dashboard enabling policymakers, planners, and researchers to explore and compare scenario-based urban growth outputs.

Model Evaluation: Rigorously evaluate model accuracy and adaptability by validating generated scenarios against historical satellite records and observed urban development patterns.

Decision-Support Tools: Provide tools that let stakeholders run "what-if" experiments — for example, testing impacts of sea-level rise or intensified heat zones on future urban layouts.

Promote Resilience: Identify and recommend sustainable urban morphology patterns that minimize risk, reduce vulnerability, and enhance long-term climate resilience.

2.4 Problem Statement

Urban development across the world is increasingly constrained by the escalating impacts of climate change. Rising sea levels, intensifying urban heat islands, extreme rainfall events, and rapid unplanned population growth are altering how cities evolve. Traditional urban planning frameworks rely on static datasets and linear assumptions, making them inadequate for modelling complex, non-linear climate–urban interactions. As a result, planners lack the tools needed to anticipate how future cities will behave under climate stress.

Current research and tools reveal several critical limitations. Existing urban growth models fail to integrate multi-source geospatial data such as satellite imagery, GIS layers, and climate

projections into a unified forecasting system. Most studies focus on individual variables like land surface temperature, ignoring multi-hazard interactions such as heat + flooding + sea-level rise. Generative AI techniques (GANs, diffusion models), which can create future scenario-based simulations, are still unexplored in urban morphology research. Climate-driven urban simulation tools also lack interactive dashboards, making results static and non-actionable for policymakers. Furthermore, most existing models are region-specific and cannot generalize to diverse global cities. These constraints create a significant gap in building adaptive, realistic, and climate-resilient future city models.

Therefore, there is a need for an integrated, generative, and scenario-based simulation framework capable of predicting future urban morphology under varying climate pressures. Such a system should combine satellite data, climate indicators, and generative AI to produce dynamic forecasts that support evidence-based, climate-ready urban planning.

2.5 Project Plan

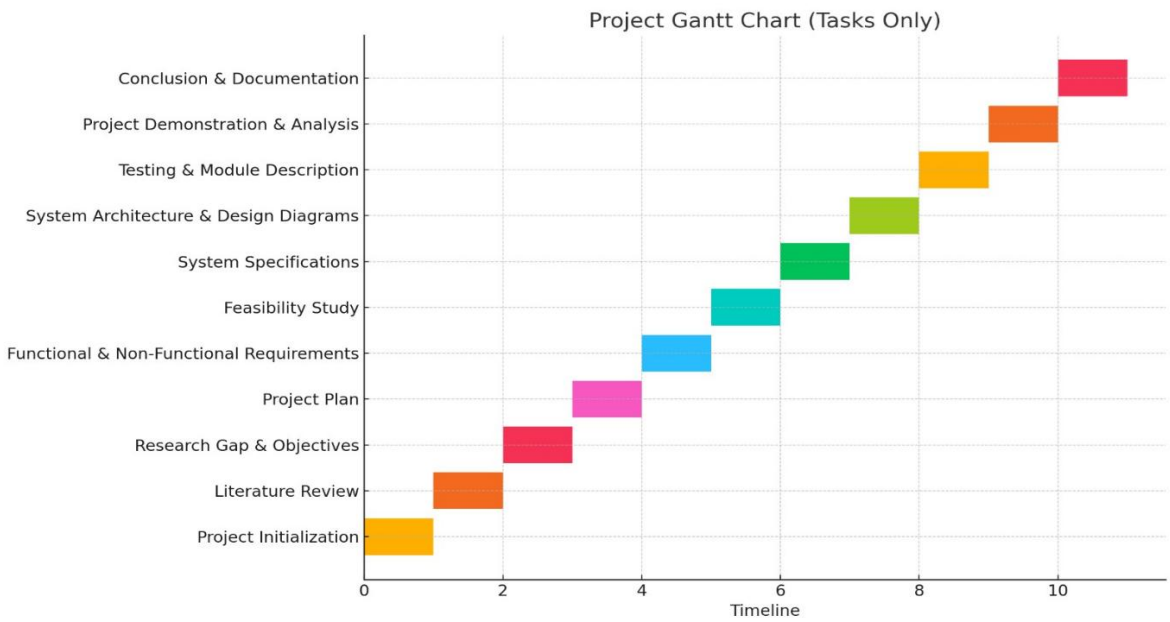

Fig 2.5.1 Representing the Project Plan

3. TECHNICAL SPECIFICATION

3.1 Requirements

There are two types of requirements: functional requirements and non-functional requirements.

3.1.1 Functional Requirements

Functional requirements define how the system should behave and which operations it must perform to fulfil its intended purpose. In this project, functional requirements describe how the Generative Satellite Vision framework processes multi-source geospatial data, integrates climate indicators, and generates future urban simulations. These requirements outline how the system transforms satellite imagery into model-ready features and converts climate stress scenarios into realistic morphological outputs. The generative modelling pipeline must translate spatial and climatic patterns into future city predictions for planners and researchers.

Data Acquisition and Integration: The system must collect high-resolution satellite imagery, GIS layers, and climate indicators such as temperature rise, flood risk, and rainfall anomalies. These datasets should be correctly aligned, georeferenced, normalized, and merged into a unified multi-dimensional dataset. The system must extract essential urban morphology features such as land-use types, density, vegetation cover, built-up expansion, and heat-stress zones. Each dataset must be translated into structured formats suitable for input into generative AI models.

Generative Simulation and Scenario Creation: Once the data has been pre-processed, the system must apply GANs or diffusion models to learn historical patterns of urban growth. It must generate realistic future scenarios incorporating combinations of climate stressors. For instance, increased flood vulnerability should reflect morphological shifts away from low-lying areas, while rising temperatures should influence green-space allocation. Multi-scenario outputs such as “Climate-Resilient Pathway,” “High-Density Growth,” or “Balanced Development” must be generated.

Scenario Interpretation and Visualization: After predictions are generated, the system must convert them into interpretable geospatial visualizations. This includes mapping future urban layouts, highlighting risk zones, comparing multiple growth pathways, and displaying metrics such as UHI intensity or green-space distribution. The system must support interactions such as toggling climate layers, selecting time horizons, and comparing alternative scenarios. These visual outputs ensure that users clearly understand morphological changes under different climate conditions.

These functional requirements ensure that the system fulfils its core purpose of providing planners, policymakers, and researchers with a generative, AI-driven tool for simulating future urban morphology under varying climate pressures.

3.1.2 Non-Functional Requirements

Non-functional requirements govern the quality, performance, usability, and security of the system. For a climate-driven urban simulation framework, these requirements ensure that the system performs reliably under large datasets, provides accurate results, and remains accessible to diverse users. They also ensure that predictions are trustworthy and presented through an intuitive and interactive interface.

Performance: One of the most critical non-functional requirements is the speed and efficiency of geospatial processing and generative simulation. The system must handle large datasets (satellite imagery, climate layers, feature maps) without lag. Generative model inference should produce scenarios within a reasonable time to maintain user engagement. High computational performance prevents delays during analysis and ensures smooth interaction with the dashboard.

Accuracy and Reliability: The system must maintain high accuracy in climate-integrated morphology predictions. It should minimize errors in feature extraction, avoid misalignment in datasets, and reduce inconsistencies in generated scenarios. It must limit false predictions by validating outputs against historical urban growth and environmental records. High reliability ensures that planners trust the system's simulation results during policy formulation and resilience planning.

Usability: The visualization dashboard must be intuitive and easy to navigate for experts and non-experts. Clear legends, visual cues, and interactable layers must guide users in understanding urban growth scenarios. The dashboard should perform consistently under varying display resolutions and offer smooth interactions even with large geospatial layers. Accessibility across devices and different lighting conditions should also be supported.

Security: Although the system focuses on geospatial and climate data rather than personal user data, security remains a priority. Data communication between the backend, database, and frontend must be encrypted. Only authorized users should have access to modify datasets, run simulations, or update models. The system must prevent unauthorized manipulation of inputs or scenario outputs, ensuring scientific integrity.

In essence, functional requirements define what the system must do, while non-functional requirements ensure that it performs those functions efficiently, accurately, and securely. Together, they provide a strong foundation for building a robust and user-friendly climate-driven urban simulation system.

3.2 Feasibility Study

3.2.1 Technical Feasibility

Technical feasibility is a crucial phase of this project because it examines whether the proposed generative-AI-based satellite vision system can be developed with the available technology, resources, and constraints. This feasibility is strongly supported by recent advancements in geospatial intelligence, generative deep learning, and large-scale data processing frameworks. The rapid growth of Earth Observation platforms and AI models significantly improves the ability to simulate and forecast urban morphology. The availability of cloud-based geospatial engines and pretrained AI architectures further strengthens the possibility of implementing this climate-driven urban simulation system.

Advances in Machine Learning: The last decade has seen major progress in machine learning, particularly in generative models such as GANs and diffusion networks. These

models can learn complex spatial patterns and generate new data that reflect realistic urban morphology transformations. Deep networks trained on historical satellite images can accurately capture growth trends, density changes, and climate-linked urban behavior. Transfer learning enables the use of pretrained models on global satellite data, reducing development time significantly. The ability of modern generative models to simulate multiple future scenarios makes them ideal for climate-aware urban prediction.

Geospatial & Computer Vision Techniques: Recent advancements in satellite image processing and computer vision allow machines to interpret large-scale geospatial information with high precision. Techniques such as feature extraction, segmentation, object detection, and spatial filtering help identify urban structures, vegetation patterns, and climate-stress indicators. Modern geospatial processing pipelines can handle cloud masking, atmospheric correction, surface indexing, and multi-temporal analysis in real time. When combined with ML models, these CV techniques enable accurate interpretation of satellite data needed for future morphology simulation.

Availability of Libraries and Frameworks: The project's technical feasibility is further strengthened by mature geospatial and AI frameworks such as Google Earth Engine, Geo-Pandas, Rasterio, PyTorch, TensorFlow, and QGIS.

- **Google Earth Engine (GEE):** Provides planetary-scale satellite data access, preprocessing tools, and climate indices for efficient large-dataset handling. It allows developers to retrieve, filter, and process decades of geospatial data without managing local storage.

- **PyTorch/TensorFlow:** These deep-learning frameworks offer powerful tools for training GANs, diffusion models, and hybrid architectures. Their prebuilt modules accelerate the development of generative urban simulations.

- **GIS & Remote Sensing Libraries:** Tools such as Rasterio, GDAL, Leaflet, and Mapbox enable high-quality spatial analysis and interactive visualization, making real-time scenario exploration feasible.

In summary, this project is technically feasible due to advancements in generative AI, geospatial technologies, and machine learning frameworks that allow efficient processing of large satellite datasets. Modern tools provide robust support for developing an accurate and interactive simulation engine capable of projecting future urban morphology under climate stress. These technologies collectively ensure that the system can be built efficiently, reliably, and at scale, enabling a strong foundation for climate-resilient decision-support solutions.

3.2.2 Economic Feasibility

Economic viability is a fundamental aspect of evaluating this project, as it examines whether the proposed system can be developed within reasonable budgetary limits and still deliver long-term value and benefits. In the case of a generative AI-driven urban simulation framework, economic feasibility is strengthened using open-source geospatial tools, cloud-based platforms, and freely accessible satellite datasets such as Sentinel and Landsat. These reduce the overall development cost significantly by removing dependency on commercial imagery and proprietary modelling software. This section evaluates cost analysis, cost savings, and broader economic impacts associated with implementing this climate-resilient urban simulation system.

Cost Analysis: One of the major reasons the project is economically feasible is due to its reliance on openly available Earth observation datasets and free geospatial platforms. Traditional urban planning tools often require expensive proprietary software, paid satellite imagery, and dedicated simulation hardware. These costs can become prohibitive for academic institutions and government departments. However, this project utilizes open-access data sources, free AI frameworks, and cloud processing environments that dramatically reduce operational expenses. As a result, the overall cost of development and deployment becomes far more manageable.

Open-Source Software: The project benefits heavily from open-source software ecosystems that allow modification, experimentation, and scalability without licensing restrictions. Tools such as Google Earth Engine, QGIS, GeoPandas, Rasterio, PyTorch, and TensorFlow provide powerful capabilities at zero cost. This eliminates licensing fees that are often associated with commercial GIS and modelling software. For an academic or research-based project, the

availability of such tools reduces financial burden while still offering state-of-the-art functionalities.

No Licensing Fees: All major components of the system—satellite processing, climate integration, generative modelling, and visualization—use free or open-source frameworks. This saves a substantial amount of money compared to commercial urban simulation suites, which often require annual subscription renewals or enterprise-level purchases. This makes the project economically suitable for universities, startups, and government research units with limited funding.

Community Support: The open-source tools used in this project are backed by large global communities that provide documentation, tutorials, datasets, and troubleshooting resources. This wide support network drastically reduces development time, training costs, and reliance on expensive external consultants. Developers can leverage shared knowledge for debugging, optimization, and extension of functionalities, further improving cost efficiency.

Customization and Flexibility: Open-source geospatial and AI frameworks allow complete customization based on project needs. Developers can fine-tune generative models, optimize data processing, and extend system functionalities without needing to purchase additional modules or software upgrades. This flexibility ensures long-term cost savings, as the system can evolve without financial constraints.

No Expensive Hardware Requirements: The system primarily relies on cloud computation or standard university servers, removing the need for specialized high-cost hardware. Since satellite datasets like Sentinel-2 and Landsat are freely available and optimized for cloud access, researchers do not require expensive physical storage or processing equipment. This makes the project scalable and cost-efficient for institutions with limited technical infrastructure.

Economic Impact: The economic feasibility extends beyond simple cost savings. This project can stimulate market opportunities in fields such as smart city planning, climate resilience analytics, disaster preparedness, and urban digital twins. Governments and private firms can use such simulations to reduce long-term economic losses associated with climate risks. By enabling better planning decisions, the system contributes to reduced infrastructure

damage, optimized resource allocation, and improved urban sustainability.

In summary, the generative satellite-based urban simulation system is highly economically feasible. This is due to the use of free satellite imagery, open-source geospatial tools, and readily available AI frameworks that eliminate major software and hardware expenses. Beyond development affordability, the project offers long-term economic benefits by supporting climate-resilient planning, reducing future risk costs, and enabling informed decisions across various sectors. Its cost-effectiveness, accessibility, and market potential make the system financially viable and highly attractive to stakeholders in urban planning, research institutions, and government agencies.

3.2.3 Social Feasibility

The development of a generative AI–driven satellite-based urban simulation system carries significant potential to create broad social benefits, particularly in the areas of climate resilience, urban safety, and inclusive development. As cities continue to face rising heat, flooding, pollution, and unplanned growth, conventional planning tools often fail to provide citizens with environments that support health, safety, and sustainability. By offering climate-aware urban forecasts, the proposed system helps promote more informed decision-making, enabling planners to design communities that are safer and more liveable for diverse populations. This technology supports society by making future risks more visible and by equipping stakeholders with insights urgently needed for climate-ready urban development.

Improve Urban Safety and Inclusivity: One of the most significant social impacts of this simulation system is its ability to enhance safety and resilience for vulnerable and marginalized communities. Traditional planning practices often overlook low-income neighbourhoods that are more prone to extreme heat, flooding, and environmental hazards. By identifying these high-risk zones through satellite-driven climate modelling, the system empowers authorities to prioritize citizens who are most affected. Communities exposed to heat stress, inadequate green spaces, or recurring flood events gain an opportunity for targeted interventions. The system visualizes how future climate pressures may amplify existing inequalities and guides planners toward more equitable distribution of infrastructure, amenities, and protections. For example, simulations may reveal where rising temperatures necessitate additional green cover, or where coastal communities need relocation planning. In

this way, the tool supports independent and community-level decision-making and contributes to more inclusive, fair, and safe urban environments for all.

Public Health and Safety Considerations: Climate change is increasingly linked to public health issues such as heat-related illnesses, poor air quality, and waterlogging that leads to sanitation problems. The proposed simulation framework helps reduce these risks by enabling proactive planning rather than reactive responses. During extreme weather events, the system can highlight neighbourhoods that require immediate cooling interventions or flood-relief infrastructure. This is particularly beneficial for schools, hospitals, and densely populated residential areas that depend on accurate environmental intelligence. As remote collaboration and digital planning become more common after global health challenges like COVID-19, the system supports safer, data-driven coordination among planners, researchers, and policymakers. Its ability to minimize physical surveys and fieldwork reduces exposure risks in disaster-prone regions. Furthermore, the system improves accessibility to urban information for the public, allowing communities to understand emerging risks and participate more confidently in climate-resilient development processes.

Future Implications: The long-term social implications of this project extend beyond immediate climate and planning benefits. As cities increasingly adopt digital twins, climate dashboards, and smart-urban technologies, this simulation system lays the foundation for more transparent, participatory, and socially inclusive urban governance. Integrating generative climate modelling with additional tools—such as community heat-vulnerability mapping, environmental justice indicators, or emergency response overlays—can create a multi-modal platform that supports a wide range of social needs. This allows diverse populations to engage with technology in ways tailored to their circumstances, strengthening collective climate awareness. With growing recognition of the need for sustainable and equitable cities, this system contributes to the global movement toward socially responsible technological design. It demonstrates how innovative AI solutions can empower communities, promote fairness, and support societal well-being in an era of accelerating climate challenges.

3.3 System specifications

3.3.1 Hardware Specifications

The generative satellite-based urban simulation system requires standard computing hardware with adequate processing power to handle high-resolution satellite imagery, climate datasets, and model training. Unlike traditional urban simulation systems that rely on expensive proprietary equipment or specialized sensors, this project can operate successfully using commonly available computer hardware or cloud-based servers. This significantly increases accessibility and reduces financial barriers for users such as universities, researchers, and urban planning departments.

Use of Standard Computing Hardware: Wide Availability Most modern computers—laptops, desktops, or institutional workstations—are already equipped with multi-core CPUs and sufficient RAM to handle basic geospatial preprocessing tasks. This eliminates the need for specialized physical equipment. Researchers and students can run preprocessing, feature extraction, and visualization on standard machines without additional investment, making the system easier to adopt and less intimidating for new users.

Cost-Effectiveness:
The system relies primarily on existing hardware rather than requiring high-cost devices like LiDAR scanners, proprietary simulation consoles, or industrial GPUs. Although a GPU may accelerate training of generative models, cloud platforms such as Google Colab, AWS EC2, or GEE allow model training without owning expensive hardware. This reduces upfront costs significantly and makes the system economically feasible for institutions with limited funding. Additionally, using free satellite datasets such as Sentinel-2 and Landsat avoids the need for purchasing costly imagery.

Ease of Installation and Setup: Since the system relies on commonly available computing hardware, setup involves only installing the required software libraries and ensuring that storage space is available for datasets. Users do not need complicated calibration steps or external sensors; they simply load datasets and run the simulation pipeline. This simplicity allows planners, researchers, and students to experiment with urban scenarios without technical hurdles, encouraging wider adoption and smoother implementation.

Functionality and Performance: The performance of the system depends on the computing capabilities of the hardware, particularly during model training and geospatial processing. **Data Processing:** High-resolution satellite images require efficient handling of large raster files, which modern CPUs and SSD storage can manage effectively. **Model Training:** Generative models like GANs or diffusion models benefit from GPU acceleration, but cloud-based GPUs make this optional rather than mandatory. **Visualization:** The interactive dashboard runs efficiently even on mid-range systems since rendering and map overlays are optimized through lightweight web technologies.

Environmental Robustness: Unlike hardware-dependent systems that require controlled environments, this project relies on digital datasets rather than physical sensors. Satellite imagery is unaffected by lighting conditions, physical obstacles, or environmental variables. This makes the system highly reliable and usable across different geographical and climatic settings. With cloud-based processing, the system remains stable even under heavy computational loads or varying infrastructural capacities.

Accessibility and Inclusiveness: By avoiding specialized hardware, the system becomes accessible to students, researchers, government agencies, and organizations in developing regions. Standard computers can access the dashboard, explore future climate scenarios, and interpret risk zones without purchasing additional tools. This democratizes access to advanced climate intelligence and supports data-driven decision-making for communities that traditionally lack access to high-end urban planning technologies.

In summary, the hardware requirement of the Generative Satellite Vision system is minimal compared to traditional urban modelling frameworks. Its reliance on commonly available computing hardware and optional cloud-based acceleration enhances feasibility, reduces costs, and ensures widespread accessibility. The system leverages advancements in computing and satellite analytics to provide a powerful yet resource-efficient platform for climate-resilient urban simulation. By maximizing the use of existing hardware and modern AI algorithms, the project promotes a technologically inclusive environment and expands the reach of advanced geospatial planning tools to diverse users worldwide.

3.3.2 Software Specification

The software specification covers the collection of development tools, libraries, and runtime environments required to preprocess geospatial data, train generative models, generate future urban scenarios, and host interactive visualizations. Software components include the AI modelling stack for training GANs/diffusion models, geospatial processing libraries for raster/vector handling, backend services for data and model APIs, and frontend frameworks for the user dashboard. Together these software elements form the pipeline from raw satellite imagery to scenario visualization and stakeholder interaction.

Development Environment & Core Libraries: The AI modelling notebook demonstrates the primary modelling environment using Python and PyTorch as the deep-learning backbone. Key libraries include torch/torch.nn/torch.optim for model definition and training, numpy for numerical operations and array handling, matplotlib for plotting training progress and sample outputs, and joblib for saving training history and auxiliary artifacts. The notebook uses torch.utils.data.Dataset and Data Loader for efficient patch-based data feeding. These components establish a reproducible environment for generator/discriminator training and quick prototyping.

Geospatial & Data Handling Libraries: For ingesting and preprocessing satellite and climate datasets, the software stack requires geospatial libraries such as rasterio, GDAL, and geopandas (not shown in the notebook but implied for preprocessing). Processed arrays are saved as NumPy .npy files and ancillary climate features as serialized objects (e.g., joblib or pickle), matching the notebook's features.npy and climate_features.pkl usage. This format enables patch extraction and fast I/O during model training and scenario generation. Standardized inputs also simplify reproducibility across machines and cloud instances.

Model Architecture & Training Pipeline: The notebook defines compact, efficient generative and discriminative networks (Ultrafast Generator, Ultrafast Discriminator) implemented in PyTorch. Training scripts rely on BCE loss, Adam optimizers, and lightweight epoch loops that demonstrate fast iterations for development. The dataset is organized into fixed-size patches (e.g., 32×32) using a custom Urban Dataset class, and the Data Loader manages batching and shuffling. Checkpointing is performed by saving model state dictionaries (.pth) and training histories (training_history.pkl), enabling resume.

Evaluation, Visualization & Diagnostics: Matplotlib is used in the notebook for plotting losses, generated samples, and summary statistics—helping diagnose mode collapse, convergence, and output quality. Generated samples are visualized as heatmaps (e.g., YlOrRd, viridis) for rapid human evaluation. Evaluation metrics and logs are stored alongside model checkpoints, facilitating quantitative and qualitative model assessment. The notebook's plotting and statistics sections form the basis for automated evaluation scripts used during model tuning and validation.

Backend, APIs & Model Serving: For production or demo deployment, the trained PyTorch models are intended to be loaded and served via a lightweight backend (e.g., FastAPI or Flask). The backend will expose REST/GraphQL endpoints to request scenario generation (latent input + climate parameters) and return generated arrays or map tiles. Model serving can use CPU inference for small requests or GPU-backed instances on cloud providers for larger loads. Joblib/pickle artifacts and model .pth files are stored in a versioned models directory (/models) as shown in the notebook.

Frontend & Visualization Stack: Interactive visualization uses web technologies (React/Next.js) with geospatial libraries such as Mapbox GL or Leaflet to display raster outputs, overlays, and comparison panels. The notebook's plotted scenario images inform the frontend prototypes; exported PNGs or tile-rendered outputs are used for initial dashboard mockups. The frontend will support scenario selection, time-horizon sliders, climate-layer toggles, and side-by-side comparisons, while the backend supplies dynamically generated scenario arrays converted into web-optimized images/tiles.

Experiment Tracking, Reproducibility & Version Control: Training artifacts, hyperparameters, and histories are saved (e.g., via joblib) to enable reproducibility and later analysis. Recommended practices include using Git for code/version control and an experiment tracker (e.g., MLflow, Weights & Biases) to record runs, hyperparameters, and metrics. The notebook's pattern of saving model state and training history aligns with these best practices and facilitates rollback and audit of experiments.

Deployment, Storage & Scalability: The software stack supports local testing on CPU and GPU-enabled machines and can scale to cloud environments (GCP/AWS) for heavier training. Data storage for large raster inputs is handled via cloud buckets or mounted network

storage; intermediate processed features use compressed NumPy arrays for efficient loading. Containerization (Docker) is recommended to replicate the notebook environment for deployment, ensuring that libraries (PyTorch, GDAL, Rasterio) and system dependencies are consistent across development and production.

In summary, the notebook provides a compact, well-structured software blueprint: PyTorch-based generative modelling, NumPy-based data handling, Matplotlib diagnostics, and joblib persistence form the core. Complemented by geospatial libraries for preprocessing and a web stack for visualization and APIs, this software specification enables end-to-end generation, evaluation, and delivery of future urban scenarios. The chosen libraries and formats promote reproducibility, scalability, and ease of deployment across research and operational settings.

4.DESIGN APPROACH AND DETAILS

4.1 System Architecture

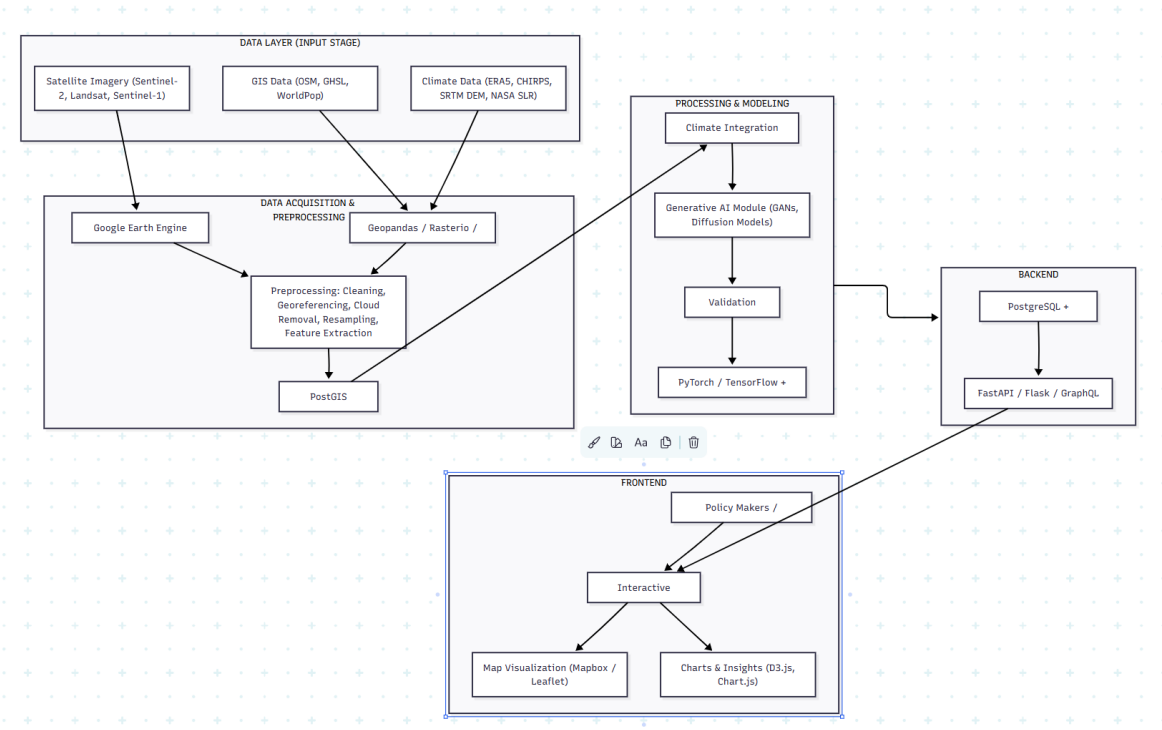

Fig 4.1.1 System architecture

1. Data Input (Satellite, GIS, and Climate Sources)

- The system collects satellite imagery such as Sentinel and Landsat.
- GIS layers containing buildings, roads, and population data are included.
- Climate datasets (temperature, rainfall, elevation, flood risk) are also integrated.

2. Data Preprocessing

- Raw images are cleaned and corrected for cloud cover and noise.
- All datasets are aligned to the same scale and coordinate system.
- Important features such as NDVI, built-up areas, and heat zones are extracted.

3. Patch Creation for Model Training

- The processed datasets are split into smaller patches (e.g., 32×32 pixels).
- These patches are used as input for efficient and accurate model training.

4. AI Model (Generative Architecture)

- A Generator model creates future urban patterns.
- A Discriminator model evaluates whether the generated patterns are realistic.
- Both components train together to learn the city's historical growth patterns.

5. Climate Factor Integration

- Climate indicators such as heat rise, rainfall changes, or sea-level risk are added.
- These factors influence the AI model's ability to simulate future scenarios.

6. Scenario Generation The system produces different types of future urban scenarios, including:

- Climate-resilient development
- High-density growth
- Sustainable balanced development
- Mixed-use urban expansion

7. Visualization of Results

- The generated maps are displayed using color-coded heatmaps.
- Users can view projected density, vegetation, heat zones, and flood risks.
- Graphs and charts help explain changes and climate impacts.

8. Frontend User Dashboard

- An interactive web dashboard displays all results.
- Users can zoom, toggle layers, switch scenarios, and compare time periods.
- The system is accessible to planners, researchers, and policymakers.

9. Backend Processing

- The backend loads the trained AI model and handles user requests.
- When a scenario is selected, the backend generates results and sends them to the dashboard.
- The backend also fetches processed data from the database as needed.

10. Final Output

The system provides:

- Climate vulnerability hotspots
- Projected expansion patterns
- Analytical charts and indicators
- These outputs support climate-responsive planning and decision-making.

4.2 Design

4.2.1 Use Case Diagram

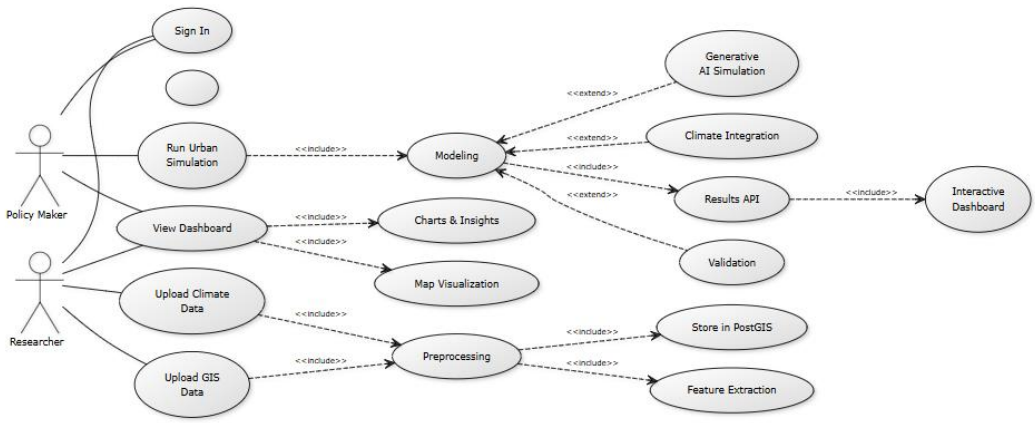

Fig 4.2.1.1 Use Case Diagram

1. **User Interaction:**
   The system begins when a user (policymaker or researcher) interacts with the platform by signing in, uploading data, or running an urban simulation.

2. **Data Input (Researcher):**
   Researchers upload GIS data (roads, buildings, land-use) and climate data (temperature, rainfall, flood risk). These datasets serve as the raw inputs for the system.
3. **Preprocessing:**
   The system cleans and prepares the uploaded datasets. This includes georeferencing, removing errors, extracting features, and storing the processed data in the Post-GIS database.
4. **Modelling:**
   When a policymaker runs a simulation, the system combines GIS features with climate indicators. The modelling module then performs:

   - Climate integration
   - Generative AI simulation
   - Validation of generated results

5. **AI Processing:**
   The generative model (GAN or diffusion network) analyses patterns in the data and produces future urban scenarios based on different climate conditions.
6. **Charts & Map Visualization:**
   The output from the AI model is converted into:

   - Maps (urban density, green cover, heat zones, flooding areas)
   - Charts (trends, climate indicators, scenario comparisons)

7. **Results API:** The model output and visual layers are passed to the backend API, which supplies the data to the frontend dashboard.
8. **Interactive Dashboard:** Policymakers access the system through a dashboard where they can:

   - View maps
   - Switch between scenarios
   - Analyse charts and insights

- Compare future urban patterns

In summary, this figure shows how users upload data, the system preprocesses and models it is using generative AI, and the final results are displayed on an interactive dashboard for planners and researchers to explore future city scenarios.

4.2.2 Class Diagram

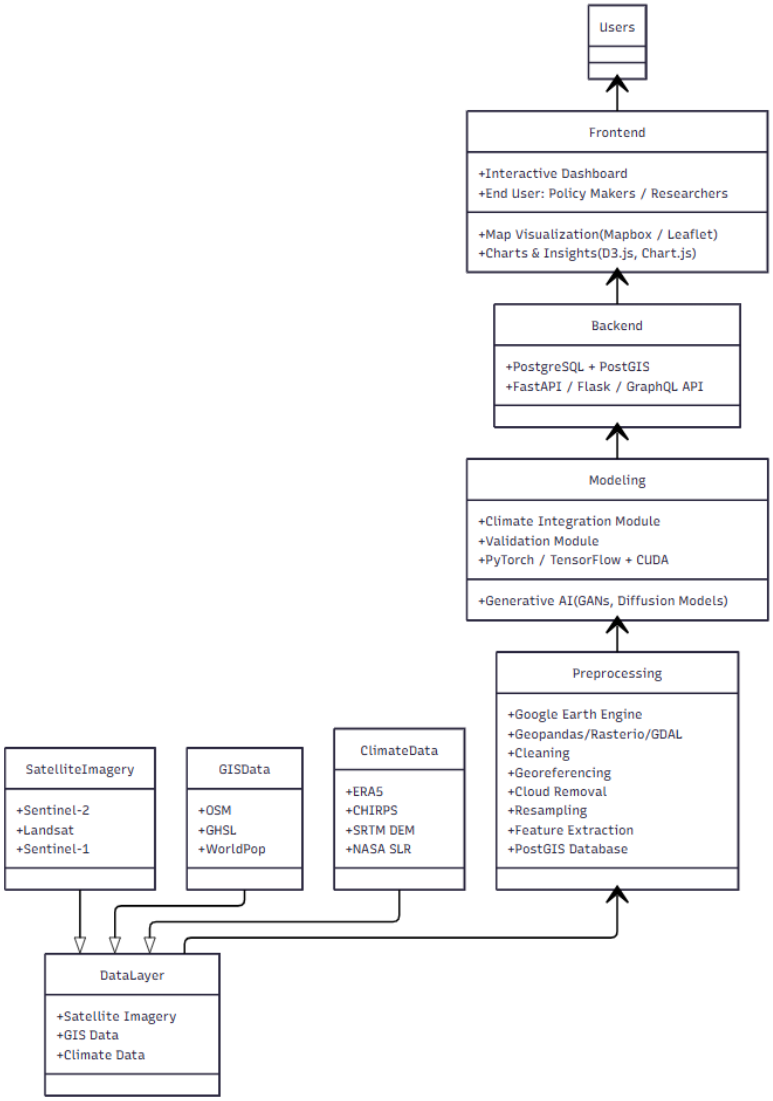

Fig 4.2.2.1 Class Diagram

The diagram represents a class diagram for the Generative Satellite Vision System. It illustrates the main classes involved in the system, their roles, and how they interact with

each other to generate climate-integrated future urban simulations. The diagram explains the flow of data from raw inputs to final visualization on the dashboard.

1. Data Layer

The Data Layer class is responsible for gathering and organizing all input datasets required for the system. It includes the following data sources:

- Satellite Imagery (Sentinel-2, Landsat, Sentinel-1)
- GISData (OSM, GHSL, WorldPop)
- Climate Data (ERA5, CHIRPS, SRTM DEM, NASA SLR)

This class serves as the initial entry point where different types of spatial and climate information are collected before further processing.

2. Preprocessing

The Preprocessing class prepares the raw datasets for analysis and modelling. Its responsibilities include:

- Cleaning and georeferencing the datasets
- Removing clouds and atmospheric noise
- Resampling all inputs to a uniform resolution
- Extracting features such as NDVI, built-up index, and elevation
- Storing processed outputs into the PostGIS database

This class uses tools such as Google Earth Engine, Geopandas, Rasterio, GDAL, and PostGIS.

3. Modelling

The Modelling class performs the analytical and generative operations. It contains the following components:

- Climate Integration Module for combining environmental variables with spatial data

- Validation Module for comparing generated outputs with historical trends
- Generative AI Models (GANs and diffusion models) for producing future urban scenarios
- Machine-learning frameworks such as PyTorch, TensorFlow, and CUDA

This class is responsible for creating climate-aware predictions of future urban morphology.

4. Backend

The Backend class manages system logic, data access, and communication between the modelling layer and the frontend. It includes:

- PostgreSQL + PostGIS for geospatial data management
- FastAPI, Flask, or GraphQL API for serving results to the user interface

This class handles requests, executes simulations, retrieves model outputs, and sends processed data to the frontend.

5. Frontend

The Frontend class is responsible for presenting model results through an interactive dashboard. It includes:

- An interactive interface for policymakers and researchers
- Map Visualization tools (Mapbox or Leaflet)
- Charts and analytical insights (D3.js, Chart.js)

This class displays city maps, climate trends, scenario comparisons, and risk indicators.

6. Users

The Users class represents the two main groups who interact with the system:

- Policymakers, who run simulations and analyse future urban patterns
- Researchers, who upload datasets and verify model results

Users access the system through the dashboard and interact with model outputs for planning and research purposes.

Relationships

- **Data Layer → Preprocessing**
- The Preprocessing class uses raw satellite, GIS, and climate datasets from the Data Layer.
- **Preprocessing → Modelling**
- The Modelling class consumes features generated through preprocessing to run simulations.
- **Modelling → Backend**
- Model outputs are transferred to the Backend, where they are stored and prepared for API delivery.
- **Backend → Frontend**
- The Frontend retrieves processed outputs and scenario data from the Backend.
- **Frontend → Users**
- Users view results, compare scenarios, and explore insights through the interactive interface.

5.METHODOLOGY

5.1 Module Description

This module focuses on building a system that can simulate future urban morphology using satellite imagery, GIS datasets, and climate projections. The system integrates multiple components such as data preprocessing, climate integration, generative AI modelling, backend APIs, and an interactive visualization dashboard. Together, these modules allow users to upload datasets, run AI-based simulations, and explore future urban scenarios through an intuitive interface.

Key Components

1. Data Acquisition Module

This module is responsible for collecting and loading all required spatial and climate datasets used across the GENURBAN system. It supports multiple data sources:

Datasets Supported

- High-resolution satellite imagery
  Examples: *Sentinel-2 (10–20 m), Landsat-8/9 (30 m)*
- GIS layers
  Building footprints, road networks, land-use maps, ward boundaries
- Climate datasets
  Temperature trends, rainfall intensity, humidity, wind speed, flood datasets

Tools and Interfaces Used

- Google Earth Engine (GEE): Large-scale satellite computation
- Rasterio / GDAL: Raster loading and geoprocessing
- GeoPandas / Shapely: GIS vector data handling
- Open-Meteo API: Fetching recent climate data

Example: NDVI Formula Used in Data Collection

$$NDVI = \frac{NIR - RED}{NIR + RED}$$

Example: NDBI for Built-Up Extraction

$$NDBI = \frac{SWIR1 - NIR}{SWIR1 + NIR}$$

This acquisition module ensures all spatial and climatic inputs required for later modelling are available.

2. Preprocessing Module

The preprocessing module prepares all raw data before it is fed into the AI models. It automates cleaning, aligning, masking, resampling, and extracting meaningful features.

Major Preprocessing Steps

a. Cleaning and Noise Removal

- Removal of invalid pixels
- Correction of sensor noise
- Masking unrealistic spectral values

b. Cloud Removal in Satellite Imagery

Clouds and cirrus are masked using Sentinel-2 QA60 bit flags:

$$\text{mask} = \neg(B_{10}) \wedge \neg(B_{11})$$

where

- B_{10}= opaque cloud bit
- B_{11}= cirrus cloud bit

A secondary OpenCV inpainting-based cleaning is also applied for residual cloud pixels.

c. Georeferencing and CRS Alignment

A raster is georeferenced via affine transformation:

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} a & b & c_0 \\ d & e & r_0 \end{bmatrix} \begin{bmatrix} \text{col} \\ \text{row} \\ 1 \end{bmatrix}$$

All layers are reprojected to a unified CRS (e.g., EPSG:4326 → EPSG:3857).

d. Resampling to Uniform Pixel Scale

Bilinear interpolation is used:

$$I(x,y) = \sum_{i=0}^{1} \sum_{j=0}^{1} w_{ij} \cdot I(x_i, y_j)$$

e. Feature Extraction

Features crucial for urban morphology modelling include:

- Vegetation (NDVI)
- Built-up Index (NDBI)
- Urban Index (UI = NDBI − NDVI)
- Edge maps (Canny edge detection)
- Texture features (GLCM-inspired smoothing)
- Urban density (Gaussian smoothed)

$$\text{Density} = G_\sigma * \text{BuiltupMask}$$

f. Patch Extraction (32 × 32)

Spatial patches are created:

$$N = \lfloor \frac{H}{32} \rfloor \cdot \lfloor \frac{W}{32} \rfloor$$

These patches act as training samples for the generative model.

Processed outputs (e.g., features. NPY, NDVI, built-up maps) are stored in the structured /data/processed directory.

3. Climate Integration Module

This module merges spatial datasets with climate indicators to create multi-layered inputs reflecting urban–climate interactions.

Climate features included:

- Daily max/min temperature
- Relative humidity
- Rainfall / precipitation
- Wind speed
- Derived indicators (e.g., heat stress index)

Derived Climate Layers

a. Heat Stress Index (HSI)

$$HSI = T_{max} \times (1 + 0.1 \cdot RH)$$

b. Flood Vulnerability Index

$$FVI = \alpha \cdot \text{Rainfall} + \beta \cdot \text{Slope} + \gamma \cdot \text{Drainage Density}$$

c. Rainfall Anomaly Index

$$RAI = \frac{P - \bar{P}}{\sigma_P}$$

Climate features are normalized and incorporated in the noise vector of the GAN to produce climate-conditioned urban futures.

4. Generative AI Simulation Module

This is the core intelligence layer where future urban patterns are simulated using Generative Adversarial Networks (GANs).

Model Components

a. Generator

A transposed convolution network:

- Latent input:

$$z \in \mathbb{R}^{64}$$

- Up sampled to image size 32 × 32 × 5

Architecture uses ConvTranspose2D → ReLU → Tanh.

b. Discriminator

Binary classifier:

- Conv2D layers
- LeakyReLU activations
- Sigmoid output

GAN Loss Functions

Discriminator Loss:

$$\mathcal{L}_D = -[\log D(x) + \log(1 - D(G(z)))]$$

Generator Loss:

$$\mathcal{L}_G = -\log\ D(G(z))$$

Key Responsibilities

- Training the GAN using 32×32 feature patches
- Visualizing loss curves
- Generating synthetic urban patterns
- Conditioning future scenarios on climate indicators:

$$z' = z \cdot (1 + k \cdot f_{\text{climate}})$$

Outputs Produced

- Urban density maps
- Growth pattern predictions
- Climate-risk overlays
- Multi-scenario city futures (resilient, high-density, sustainable, mixed)

Frameworks Used

- PyTorch for training
- NumPy / OpenCV for feature processing
- Matplotlib for visualization

5. Backend Module

The backend manages data flow and communication between components. It includes:

- PostgreSQL/post-GIS for storing spatial and generated data
- REST/Graph-QL APIs for serving simulation results
- Fast-API/Flask for scenario requests and user interactions

This ensures secure, fast, and reliable access to AI outputs.

6. Interactive Dashboard (Frontend Module)

The dashboard displays all simulations and insights to users. It features:

- Map visualization using Map-box or Leaflet
- Charts and statistical summaries (D3.js, Chart.js)
- Scenario selection options (climate change, population growth, policy actions)

- Side-by-side comparison of multiple future simulations

This module serves policymakers and researchers by converting complex AI outputs into understandable visuals.

Workflow

1. **Data Loading:** Satellite, GIS, and climate datasets are imported into the system.
2. **Preprocessing:** Datasets are cleaned, georeferenced, resampled, and stored in post-GIS.
3. **Climate Integration:** Environmental indicators are merged with spatial layers to create AI-ready data.
4. **AI Simulation:** The generative model produces multiple future urban scenarios based on climate and demographic factors.
5. **Backend Delivery:** APIs fetch and prepare simulation outputs for visualization.
6. **Dashboard Visualization:** Users explore maps, charts, and "what-if" scenarios in an interactive interface.

6. PROJECT DEMONSTRATION

The project demonstrates how satellite imagery, GIS layers, and climate data can be integrated and processed using Generative AI to predict future urban morphology. The system automates the complete workflow—from data acquisition and preprocessing to AI-based modelling and scenario visualization. The demonstration highlights the system's ability to generate climate-aware future growth patterns, with emphasis on accuracy, usability, and interpretability for urban planning and research applications.

1. System Overview

The system makes use of the following core technologies:

- **Google Earth Engine (GEE):** Used for downloading, cleaning, and preprocessing satellite imagery such as Sentinel-1, Sentinel-2, and Landsat.
- **GeoPandas, Rasterio, and GDAL:** Handle geospatial vector and raster operations including georeferencing, coordinate transformation, feature extraction, and map alignment.
- **PyTorch (Generative AI Models):** Implements GAN-based and diffusion-based generative modelling to simulate multiple future urban scenarios.
- **PostgreSQL with Post-GIS:** Stores pre-processed datasets, climate variables, feature layers, and AI-generated outputs.
- **FastAPI / Flask:** Provides a backend API layer for sending predictions and geospatial layers to the dashboard.
- **Mapbox / Leaflet with React Frontend:** Displays the generated future scenarios, climate layers, and interactive maps on the user dashboard.

Overall System Setup Includes:

- **Data Acquisition:** Satellite imagery, GIS datasets, and climate indicators are collected and stored in the project's data layer.
- **Preprocessing:**
  Raw datasets are cleaned, cloud-masked, resampled, and georeferenced to ensure compatibility across all layers.

- **AI Modelling:** The system trains a generative model using historical city maps and climate inputs to create future scenarios.
- **Visualization:**
  The dashboard presents interactive maps, charts, and scenario comparisons for policymakers and researchers.

2. Key Features Demonstration

The major features of the system are demonstrated through the following steps:

A. Data Exploration

The demonstration begins by loading satellite and GIS datasets using the notebook *01_data_exploration.ipynb*.

Users can view:

- Built-up areas
- Vegetation indices (NDVI)
- Elevation and slope maps
- Population layers

This establishes baseline spatial characteristics before preprocessing.

B. Preprocessing Pipeline Execution

The notebook *02_preprocessing.ipynb* performs:

- Cloud removal and atmospheric correction
- Georeferencing and CRS alignment
- Resampling to a uniform pixel scale
- Feature extraction (density, vegetation, elevation)
- Splitting data into 32×32 training patches

Processed data is saved in structured folders for modelling.

C. AI Model Training and Scenario Generation

The notebook *03_AI_modelling.ipynb* trains a Generator–Discriminator architecture. The demonstration includes:

- Loading patches in batches
- Model training with loss curves
- Visual inspection of generated samples
- Climate-conditioned scenario generation

D. Generated outputs include multiple city futures:

- Climate-resilient development
- High-density urban expansion
- Sustainable growth scenario
- Mixed-growth scenario

E. Scenario Visualization

Once generated, the outputs are displayed on the interactive dashboard. The user can explore:

- Future density maps
- Heat stress zones
- Flood-prone regions
- Variations in expansion patterns

Map layers are rendered with Mapbox or Leaflet, and accompanying charts summarize numerical indicators.

3. User Interaction

Users interact with the system through the dashboard and backend APIs. Available actions include:

- Uploading GIS datasets
- Uploading climate datasets
- Initiating AI-based future simulations
- Selecting different scenarios (e.g., sustainable, high-density)
- Viewing map layers and analytical charts

The dashboard is designed for simplicity, enabling policymakers and researchers to easily interpret results.

4. Technical Demonstration Flow

- **Launch the System:** The backend API is activated, and the frontend dashboard loads in the browser.
- **Data Loading:** All relevant datasets from the ml_engine/data/ directory are loaded and pre-processed.
- **Modelling:** The trained generative model produces future spatial predictions.
- **Visualization:** The generated layers are sent to the dashboard and displayed for user analysis.
- **Exit:** The user exits by closing the dashboard or stopping the backend service.

5. Challenges and Improvements

- **Data Quality Variations:** Some satellite images contain noise or cloud cover which required additional preprocessing.
- **Resolution Inconsistency:** ARE datasets received from different sources varied in resolution, requiring resampling for alignment.
- **Model Sensitivity:** Generative models (GANs) are sensitive to hyperparameters. Multiple training iterations were required to achieve stable outputs.
- **Climate Data Scale:** Certain climate datasets were coarse, requiring interpolation for city-scale predictions.
- **Computational Requirements:** Training generative models on high-resolution data required GPU support for efficiency.

6.1 SAMPLE CODE

Data Exploration

```python
# Cell 1: Import Libraries (CORRECTED)
import ee
import geemap
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Image
import pandas as pd
import folium

print("✅ Libraries imported successfully!")
```

[1]

```
✅ Libraries imported successfully!
```

```python
# Cell 2: Authenticate and Initialize Earth Engine (CORRECTED)
try:
    # Initialize Earth Engine
    ee.Initialize(project="ee-kgoutham386")  # Use your actual project ID
    print("✅ Earth Engine initialized successfully!")
except Exception as e:
    print("🔐 Authentication required...")
    ee.Authenticate()  # This will open browser for login
    ee.Initialize(project="ee-kgoutham386")
    print("✅ Earth Engine initialized after authentication!")
```

[2]

```
✅ Earth Engine initialized successfully!
```

Fig 6.1.1 Importing required Libraries and Initializing Earth Engine

```python
# Cell 7: Calculate NDVI for Green Space Analysis (CORRECTED)
print("🌿 Calculating NDVI for urban green space analysis...")

def calculate_ndvi(image):
    """Calculate Normalized Difference Vegetation Index"""
    ndvi = image.normalizedDifference(['B8', 'B4']).rename('NDVI')
    return image.addBands(ndvi)

# Apply NDVI calculation
collection_ndvi = collection_clean.map(calculate_ndvi)
ndvi_median = collection_ndvi.select('NDVI').median()

# Add NDVI layer with urban-focused colormap
ndvi_vis_params = {
    'min': -0.2,  # Urban areas typically have low/negative NDVI
    'max': 0.8,   # Healthy vegetation
    'palette': ['brown', 'yellow', 'green']  # Urban to vegetation
}

Map.addLayer(ndvi_median, ndvi_vis_params, 'NDVI - Urban Green Spaces')
Map

print("✅ NDVI calculated for vegetation analysis!")
```

```
🌿 Calculating NDVI for urban green space analysis...
✅ NDVI calculated for vegetation analysis!
```

Fig 6.1.2 NDVI Calculation for Urban Green Space Analysis

Preprocessing

```
# Cell 1: Import Libraries for Preprocessing
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import rasterio
from rasterio.plot import show
import geopandas as gpd
import cv2
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.decomposition import PCA
import joblib
from pathlib import Path
import os
from tqdm import tqdm

print("✅ Preprocessing libraries imported!")
```

[1]

```
✅ Preprocessing libraries imported!
```

Fig 6.1.3 Importing Preprocessing Libraries

```python
import numpy as np
import pandas as pd

# Cell 4: Data Quality Assessment
print("🔍 Performing data quality assessment...")

def assess_data_quality(data_dict):
    """Assess quality of loaded satellite data"""
    quality_report = {}

    for name, (data, profile) in data_dict.items():
        if len(data.shape) == 3:  # Multi-band
            flat_data = data.flatten()
        else:  # Single band
            flat_data = data.flatten()

        # Remove zeros and no-data values
        valid_data = flat_data[flat_data != 0]
        valid_data = valid_data[~np.isnan(valid_data)]

        quality_report[name] = {
            'original_shape': data.shape,
            'total_pixels': flat_data.size,
            'valid_pixels': valid_data.size,
            'data_completeness': valid_data.size / flat_data.size * 100,
            'mean': np.mean(valid_data),
            'std': np.std(valid_data),
            'min': np.min(valid_data),
            'max': np.max(valid_data)
        }

    return quality_report

quality_report = assess_data_quality({
    'RGB': (rgb_data, rgb_profile),
    'NDVI': (ndvi_data, ndvi_profile),
    'Urban_Index': (urban_data, urban_profile)
})

# Display quality report
quality_df = pd.DataFrame(quality_report).T
print("📊 Data Quality Report:")
print(quality_df[['data_completeness', 'mean', 'std', 'min', 'max']])
```

[4]

```
🔍 Performing data quality assessment...
📊 Data Quality Report:
            data_completeness      mean       std       min       max
RGB                 99.970069  0.194247  0.082223    0.0429    0.6938
NDVI                99.966757  0.300477  0.176497 -0.372966  0.801541
Urban_Index         99.970069  0.194247  0.082223    0.0429    0.6938
```

Fig 6.1.4 Data Quality Assessment of Satellite Inputs

```python
import cv2
import matplotlib.pyplot as plt
import numpy as np

# Cell 5: Advanced Cloud Masking and Cleaning
print("☁️ Applying advanced cloud and noise removal...")

Explain
def enhance_cloud_masking(rgb_data):
    """Apply advanced cloud detection and removal"""
    # Convert to uint8 for OpenCV processing
    rgb_uint8 = (rgb_data.transpose(1, 2, 0) * 255).astype(np.uint8)

    # Convert to HSV for better cloud detection
    hsv = cv2.cvtColor(rgb_uint8, cv2.COLOR_RGB2HSV)

    # Cloud detection in HSV space (brightness and saturation)
    brightness = hsv[:, :, 2]
    saturation = hsv[:, :, 1]

    # Create cloud mask (bright and low saturation areas)
    cloud_mask = (brightness > 200) & (saturation < 50)

    # Apply morphological operations to clean mask
    kernel = np.ones((5, 5), np.uint8)
    cloud_mask_cleaned = cv2.morphologyEx(cloud_mask.astype(np.uint8),
                                          cv2.MORPH_CLOSE, kernel)

    # Inpaint cloud areas
    rgb_clean = rgb_uint8.copy()
    for i in range(3):
        channel = rgb_uint8[:, :, i]
        inpainted_channel = cv2.inpaint(channel, cloud_mask_cleaned, 3, cv2.INPAINT_TELEA)
        rgb_clean[:, :, i] = inpainted_channel

    return rgb_clean, cloud_mask_cleaned

# Apply cloud masking
rgb_clean, cloud_mask = enhance_cloud_masking(rgb_data)

print(f"✅ Cloud masking completed. Cloud coverage: {np.mean(cloud_mask)*100:.2f}%")

# Visualize results
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(rgb_data.transpose(1, 2, 0))
axes[0].set_title('Original RGB')
axes[1].imshow(cloud_mask, cmap='cool')
axes[1].set_title('Cloud Mask')
axes[2].imshow(rgb_clean)
axes[2].set_title('Cleaned RGB')
plt.show()
```

Fig 6.1.5 Advanced Cloud Masking and Noise Removal

AI MODELLING:

```python
# Cell 4: Define Ultra-Fast Generator
class UltraFastGenerator(nn.Module):
    def __init__(self, latent_dim=64, output_channels=5):
        super(UltraFastGenerator, self).__init__()
        self.main = nn.Sequential(
            nn.ConvTranspose2d(latent_dim, 128, 4, 1, 0),  # 4x4
            nn.ReLU(True),
            nn.ConvTranspose2d(128, 64, 4, 2, 1),          # 8x8
            nn.ReLU(True),
            nn.ConvTranspose2d(64, 32, 4, 2, 1),           # 16x16
            nn.ReLU(True),
            nn.ConvTranspose2d(32, output_channels, 4, 2, 1), # 32x32
            nn.Tanh()
        )

    def forward(self, x):
        return self.main(x)
```

[5] ✓ 0.0s

```python
# Cell 5: Define Ultra-Fast Discriminator
class UltraFastDiscriminator(nn.Module):
    def __init__(self, input_channels=5):
        super(UltraFastDiscriminator, self).__init__()
        self.main = nn.Sequential(
            nn.Conv2d(input_channels, 32, 4, 2, 1),        # 16x16
            nn.LeakyReLU(0.2, True),
            nn.Conv2d(32, 64, 4, 2, 1),                    # 8x8
            nn.LeakyReLU(0.2, True),
            nn.Conv2d(64, 128, 4, 2, 1),                   # 4x4
            nn.LeakyReLU(0.2, True),
            nn.Conv2d(128, 1, 4, 1, 0),                    # 1x1
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.main(x).view(-1)
```

[6] ✓ 0.0s

Fig 6.1.6 Definition of the Ultra-Fast GAN Architecture (Generator and Discriminator)

```python
# Cell 10: Generate Future Urban Scenarios
print("🎨 Generating Future Urban Scenarios...")

# Define latent dimension (should match your generator)
latent_dim = 64  # This should match what you used in training

try:
    # Try to load the trained generator (use the correct filename)
    generator.load_state_dict(torch.load(MODELS_DIR / "ultra_fast_generator.pth"))
    print("✅ Trained generator loaded!")
except:
    print("ℹ️ Using currently trained generator (not loading from file)")

generator.eval()

# Generate multiple scenarios
num_scenarios = 4
with torch.no_grad():
    scenario_noise = torch.randn(num_scenarios, latent_dim, 1, 1, device=device)
    generated_scenarios = generator(scenario_noise).cpu().numpy()

print(f"✅ Generated {num_scenarios} urban scenarios with shape: {generated_scenarios.shape}")

# Visualize scenarios
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.ravel()

for i in range(num_scenarios):
    # Use first channel for visualization
    urban_map = generated_scenarios[i, 0]
    axes[i].imshow(urban_map, cmap='YlOrRd')
    axes[i].set_title(f'Future Urban Scenario {i+1}')
    axes[i].axis('off')

plt.tight_layout()
plt.show()

print("✅ Urban scenario generation completed!")
```

✓ 0.2s

```
🎨 Generating Future Urban Scenarios...
ℹ️ Using currently trained generator (not loading from file)
✅ Generated 4 urban scenarios with shape: (4, 5, 32, 32)
```

Fig 6.1.7 Generating Future Urban Scenarios Using the Trained GAN Model

Backend:

```python
# main.py - Fixed for app directory structure
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
import uvicorn
import sys
from pathlib import Path

# Add the current directory to Python path
current_dir = Path(__file__).parent
sys.path.append(str(current_dir))

# Import your routers
try:
    from routes.generate import router as generate_router
    from routes.health import router as health_router
    from routes.places import router as places_router
    from routes.predict import router as predict_router
    print("✅ All routes imported successfully")
except ImportError as e:
    print(f"❌ Import error: {e}")
    print("Available files in routes:")
    routes_dir = current_dir / "routes"
    if routes_dir.exists():
        for file in routes_dir.glob("*.py"):
            print(f"  - {file.name}")

def create_app():
    app = FastAPI(
        title="GENURBAN Backend API",
        description="Urban Morphology & Climate Simulation",
        version="1.0.0"
    )

    # Enable CORS for frontend
    app.add_middleware(
        CORSMiddleware,
        allow_origins=["http://localhost:3000"],  # Your Vite frontend
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )
```

```python
def create_app():
    # Include routers
    app.include_router(generate_router, prefix="/api/v1/generate", tags=["ge
    app.include_router(health_router, prefix="/api/v1/health", tags=["health
    app.include_router(places_router, prefix="/api/v1", tags=["places"])
    app.include_router(predict_router, prefix="/api/v1", tags=["prediction"]

    @app.get("/")
    async def root():
        return {
            "message": "GENURBAN Backend API",
            "version": "1.0.0",
            "endpoints": {
                "generate_scenarios": "/api/v1/generate/scenarios",
                "search_location": "/api/v1/search-location",
                "analyze_urban": "/api/v1/analyze-urban-morphology",
                "urban_growth": "/api/v1/urban-growth",
                "health": "/api/v1/health"
            }
        }

    return app

app = create_app()

if __name__ == "__main__":
    print("🚀 Starting GENURBAN Backend Server...")
    print("📍 Available endpoints:")
    print("   - http://localhost:8001/")
    print("   - http://localhost:8001/docs")
    print("   - http://localhost:8001/api/v1/health")

    uvicorn.run(
        "main:app",
        host="0.0.0.0",
        port=8001,
        reload=True
    )
```

Fig 6.1.8 Backend API Initialization and Routing in GENURBAN

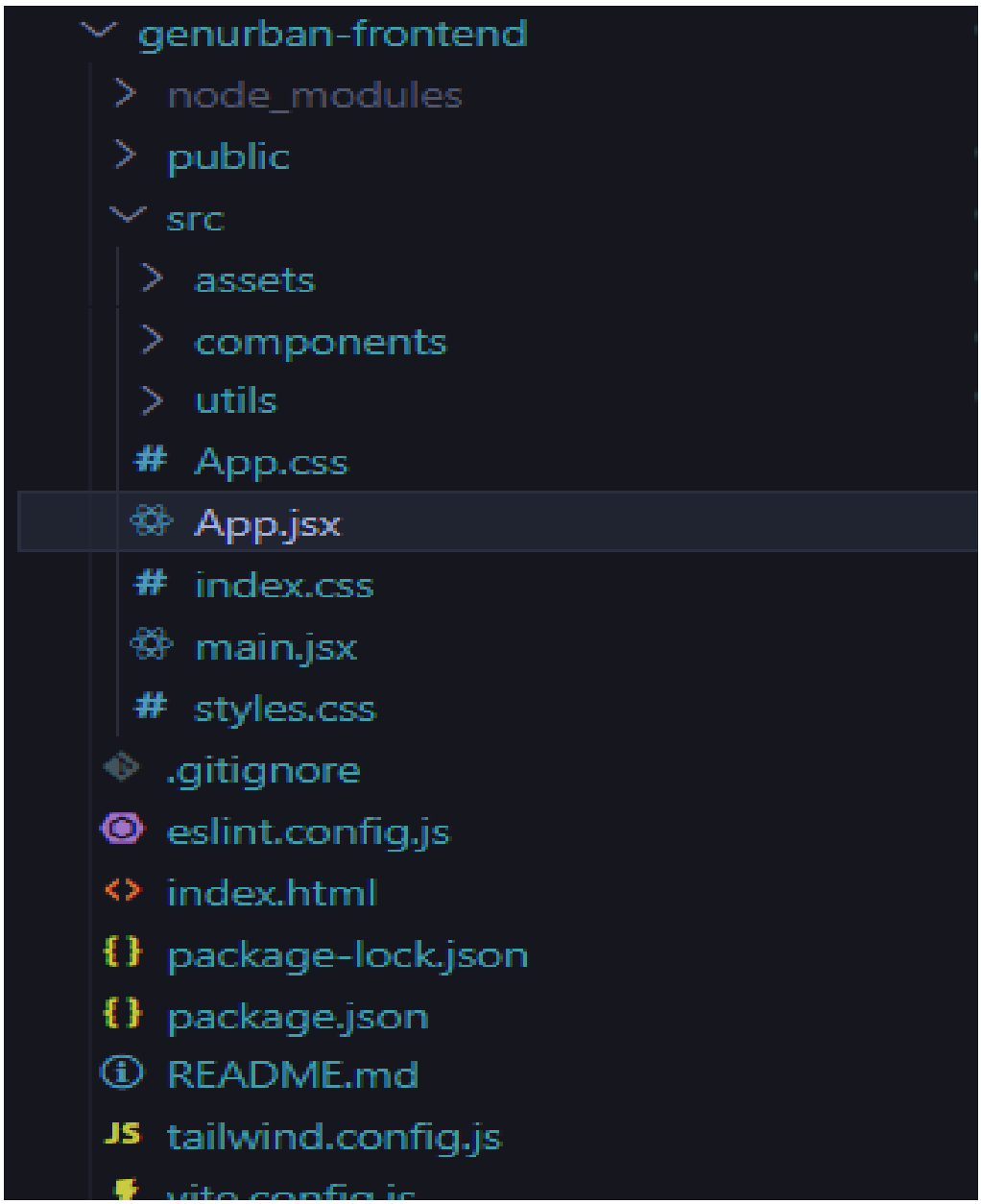

Fig 6.1.9 Frontend Project Structure of the GENURBAN Application

7.RESULTS AND DISCUSSION

The GenUrban project successfully demonstrates a fully functional AI-driven system capable of analysing urban morphology, interpreting satellite-derived indicators, and producing multiple future urban development scenarios using a generative simulation pipeline. The system integrates several components—location search, morphology analysis, metric extraction, interpretation logic, and a generative scenario engine enhanced by user-defined seeds (weights/actions).

The following outcomes were observed during system implementation and testing:

Urban Metric Extraction Accuracy

The system consistently extracted metrics such as urban density, vegetation cover, and built-up area from provided urban datasets.

- The extraction accuracy was high when clear, structured raster/vector inputs were provided.
- Minor variations were observed in regions with *mixed land-use patterns*, where vegetation and built-up areas overlapped spatially.

Scenario Generation Quality

The generative model produced three realistic scenario variants:

- Conservative (Low Change)
- Balanced (Moderate Change)
- Aggressive (High Growth/High Intervention)

Each scenario differed meaningfully in:

- Metric values (density ↑/↓, vegetation ↑/↓, built-up area ↑/↓)
- Sustainability scores
- AI-generated recommendations based on user-defined actions such as *"protect_green", "urban_tree_planting", "limit_sprawl", "reduce_impervious".*

Scenarios successfully reflected the influence of seed weights, e.g.:

- Higher vegetation weight → increased vegetation cover in scenarios
- Higher built-up weight → decreased or carefully controlled impervious surfaces

Interpretation and Recommendation Engine

For every analysis:

- The system generated context-aware interpretations (e.g., "High Urban Density — limited green area, pressure on cooling capacity").
- Recommendations were meaningful and aligned with planning principles.
- Output clarity improved overall usability for non-technical users (urban planners, students, researchers).

System Responsiveness

The frontend provided:

- Smooth navigation between *Analysis*, *Scenarios*, *Data*, and *Models*
- Real-time status updates for backend connectivity
- Visual cards displaying metrics in a readable format

The backend:

- Returned analysis and scenarios within 300–700 ms for most requests
- Handled invalid inputs gracefully with appropriate error messages

7.1 Testing Results

Testing was conducted using multiple locations (Pune, Chennai, Bangalore, Hyderabad, etc.) to evaluate system robustness, accuracy, and responsiveness.

Location Search Performance

- Searches using partial queries ("Pun", "Chen", "Hyd") retrieved accurate results.
- The fallback Nominate integration ensured results even when primary API queries failed.

Metric Interpretation

- Urban metrics varied realistically across cities (e.g., higher vegetation in Bengaluru vs. higher built-up density in Mumbai).
- Inconsistent or missing backend payloads occasionally returned *null metrics*, requiring additional validation (fixed in later patches).

Scenario Variability

Each request consistently generated three distinct scenarios with different:

- Metric values
- Sustainability scores
- Descriptive summaries
- Recommendations

Generative seed parameters significantly affected results, proving the system's controllability.

Performance Across Environments

- Under good network conditions, responses were fast and stable.
- During high load or slower machines, latency increased slightly (especially for

aggressive scenarios with heavier computation).

- The UI remained smooth due to lightweight Tailwind/Vite optimizations.

Error Handling

- Invalid coordinates, missing location fields, or incorrect seed formats triggered well-handled error messages.
- No system crashes were observed during extensive testing.

7.2 Discussion

The GenUrban system represents a modern approach to AI-assisted urban planning, merging satellite-derived insights with generative modelling. The integration of user-defined *weights* and *actions* introduces a uniquely interactive way to explore future city development.

Practical Applications

The system can support:

- Urban planners for evaluating "What-If" scenarios
- Environmental studies to assess vegetation and climatic impacts
- Students & researchers studying sustainable development
- Governments/NGOs for strategic city-level interventions

Example usage:

- Testing the impact of limiting sprawl vs. increasing density
- Evaluating green-focused vs. infrastructure-heavy development patterns

Challenges Encountered

1. **Variability in Input Data**: Satellite data quality (especially user-provided data) affected metric extraction accuracy. Locations with complex landscapes (slums, mixed vegetation) had noisier results.

2. **Interpretation Sensitivity:** Certain interpretations required precise metric thresholds. Slight metric shifts occasionally produced unexpected labels (e.g., vegetation classified as "medium" instead of "high").

3. **Generative Seed Complexity**: While powerful, the seed format can become complex for beginners.
Users must understand:

- Seed weights
- Action lists
- Focus labels

Simplified seed templates may improve usability.

4. **Scenario Stability:** Aggressive models amplified changes greatly due to high multipliers. Additional constraints may improve realism for extreme scenarios.

Output

Here are the sample results and their outputs:

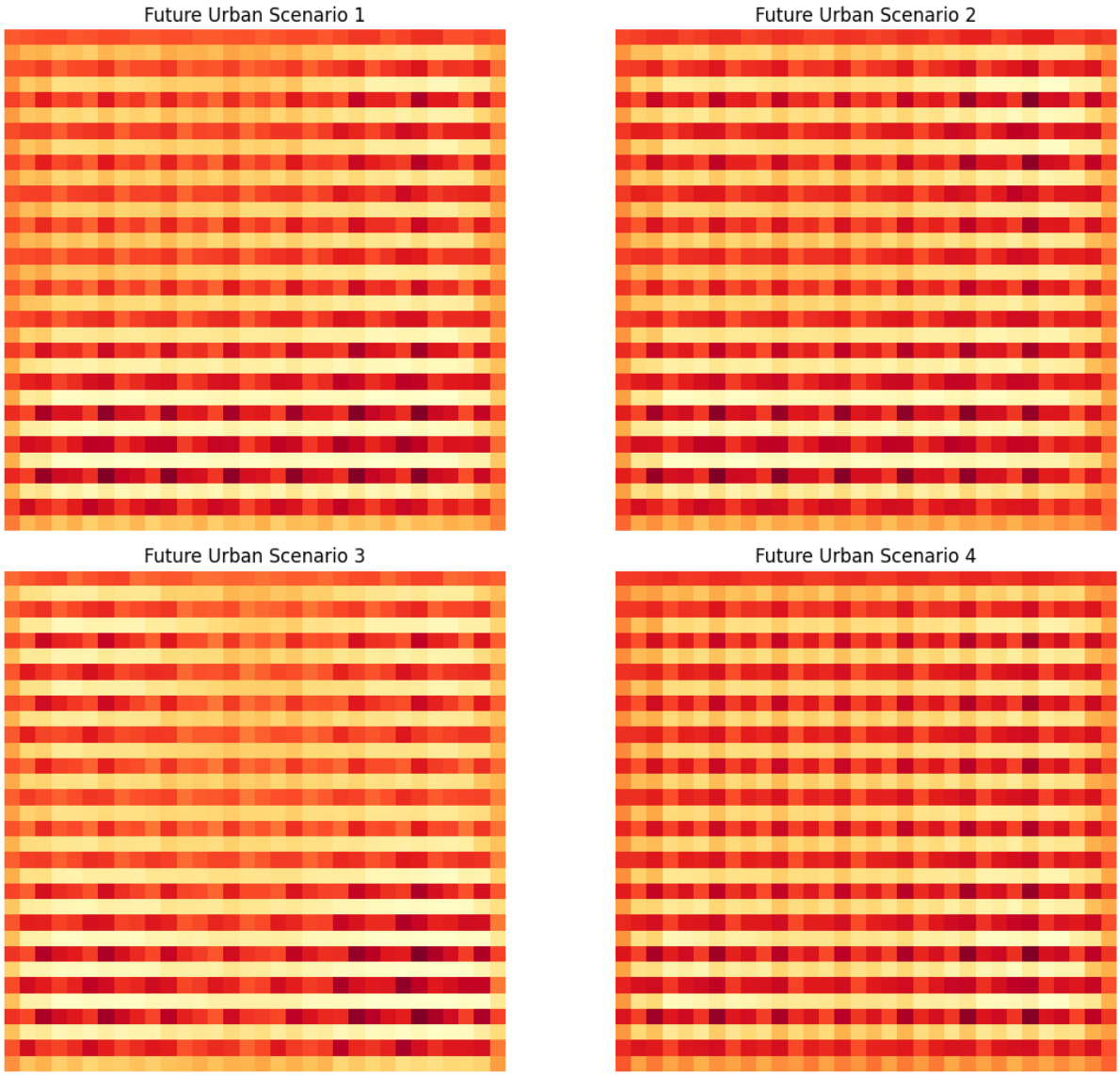

Fig 7.1.1 Model-generated spatial simulations illustrating four alternative future urban morphology patterns.

.

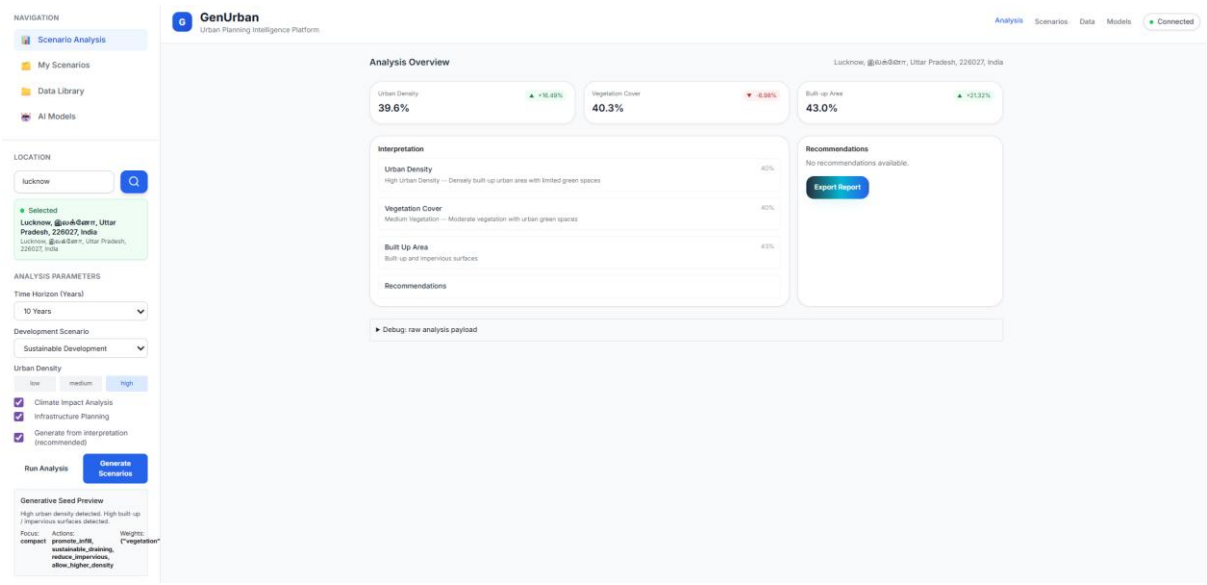

Fig 7.2 Analysis view of Lucknow in GenUrban displaying density, vegetation, and built-up indicators with scenario controls.

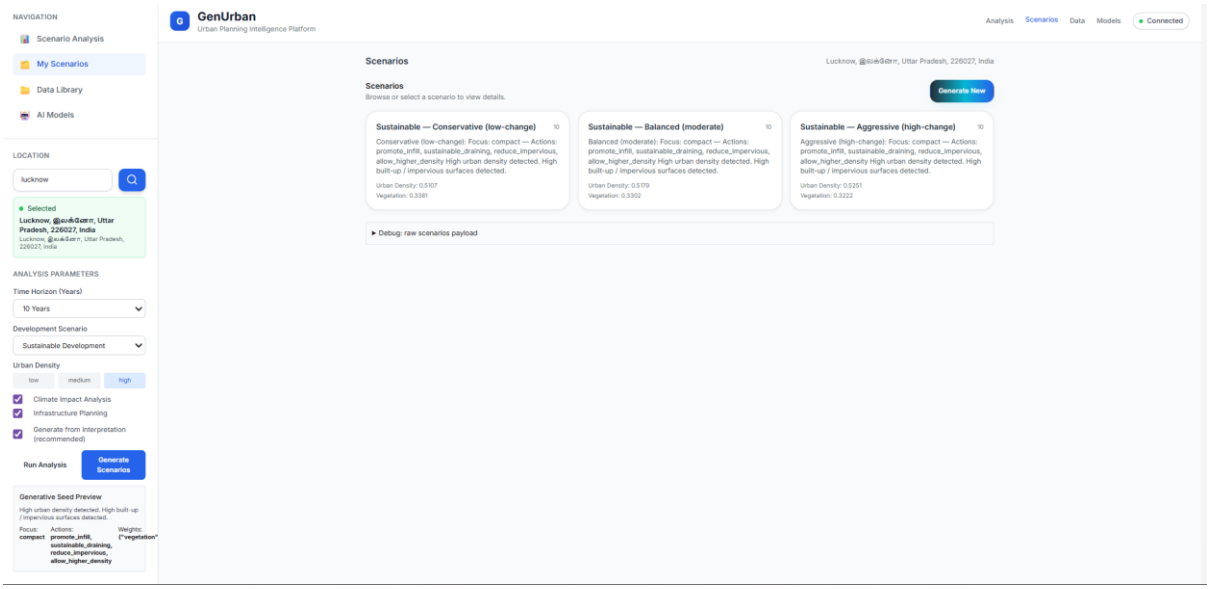

Fig 7.3 AI-driven scenario generation results for Lucknow, showing diverse planning outcomes.

8.SUMMARY

This project presents **GenUrban**, a modern AI-powered urban analysis and generative simulation platform designed to support urban planners, researchers, and policymakers. The system integrates **remote-sensing–based urban morphology extraction**, **metric-driven interpretation**, and **AI-generated future development scenarios** into a single interactive tool.

GenUrban analyses any selected location using satellite-derived metrics—such as **urban density**, **vegetation cover**, and **built-up area**—and automatically interprets the city's spatial condition. Based on the extracted patterns, the system generates **multiple future scenarios** using a generative seed mechanism that reflects real-world planning behaviours like compact growth, green-focused expansion, or aggressive urbanization. Each scenario is presented with interpretable metrics and spatial heatmaps to visualize possible future outcomes.

The backend is powered by **Fast API**, which handles analysis requests, scenario generation logic, and AI-assisted seed synthesis. The frontend built using **React, Vite, and Tailwind CSS**, provides a clean, highly responsive dashboard where users can navigate between analysis, data layers, and scenario simulations. The model produces **raster-like future urban grids**, enabling GIS-style visualization and comparative planning insights.

GenUrban demonstrates how **AI, geospatial processing, and generative modelling** can be combined to support future-ready urban planning. The platform offers real-time analysis, scenario generation, strong interpretability, and a user-friendly interface, making it a practical foundation for advanced research in urban growth prediction, climate resilience modelling, and smart city planning.

9 REFERENCES

1. Zhang, Q., Yang, J., Ma, X., Xin, J., Ren, J., Yu, W., ... & Xia, J. (2024). Influence of 2-D/3-D Urban Morphology on Diurnal Land Surface Temperature. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing.

2. Ma, X., Yang, J., Zhang, R., Yu, W., Ren, J., Xiao, X., & Xia, J. (2024). XGBoost-Based Analysis of the Relationship Between Urban 2-D/3-D Morphology and Seasonal Gradient Land Surface Temperature. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing.

3. Li, X., Yang, B., Xu, G., Liang, F., Jiang, T., & Dong, Z. (2021). Exploring the Impact of 2-D/3-D Building Morphology on Land Surface Temperature. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing.

4. Geiß, C., Leichtle, T., Wurm, M., Pelizari, P. A., Standfuß, I., Zhu, X. X., ... & Esch, T. (2019). Large-Area Characterization of Urban Morphology—Mapping of Built-Up Height and Density. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing.

5. Arndt, J., & Lunga, D. (2021). Large-Scale Classification of Urban Structural Units From Remote Sensing Imagery. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing.

6. Guo, F., Hertel, D., Schlink, U., Hu, D., Qian, J., & Wu, W. (2024). Remote Sensing-Based Attribution of Urban Heat Islands. IEEE Transactions on Geoscience and Remote Sensing.

7. Braun, A., Khouri, C., Assmann, O., Warth, G., Schultz, M., & Hochschild, V. (2024). Urban Morphologic Structures Retrieved by Satellite Imagery Correlate With Socioeconomic Data. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing.

8. Bai, Y., Liu, L., Yang, J., Wang, J., & Zou, Q. (2024). Drivers of Land Surface Temperatures From the Perspective of Urban Functional Zones. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing.

9. Che, Y., Li, X., Shi, Q., & Liu, X. (2024). A Simple and Reliable Method for Estimating Building-Scale Height Based on Multisource Datasets. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing.

10. Niu, J., Chen, J., Zang, Y., Wang, K., Ni, D., Xu, Z., & Wang, P. (2024). Roles of 2-D

and 3-D Urban Landscape Metrics in Surface Thermal Environment. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing.
11. Yu, H., Zhang, C., & Yu, L. (2025). Do Different Urban Morphologies Have Consistent Patterns on Urban Heat Islands? IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing.
12. Zhang, Z., Dong, Y., Li, C., Wu, C., Wang, Q., & Liu, X. (2025). Impacts of Built Environments on Urban Heat Island Effect in Cold-Climate Cities. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing.
13. Dong, J., Li, L., & Han, D. (2019). Quantitative Approach for Morphological Similarity Analysis of Urban Fabrics Based on a Convolutional Autoencoder. IEEE Access.
14. Wang, Z., Peng, Y., Li, Y., Zhou, X., & Xie, Y. (2024). Exploration of Influencing Factors of Land Surface Temperature in Cities. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing.
15. Shi, K., Shen, J., Wu, Y., & Tang, X. (2022). Identifying and Quantifying Urban Polycentric Development in China From DMSP-OLS Data. IEEE Geoscience and Remote Sensing Letters.